CPN and A Comparative Analysis vs BPN

Introduction

Counter-Propagation Networks (CPN)

The Counter-Propagation Network (CPN) is a powerful hybrid architecture that combines two distinct types of learning: Unsupervised Competitive Learning and Supervised Learning. While the Back-propagation (BPN) model from our previous project relied on gradient descent to minimize errors across multiple hidden layers, the CPN adopts a "clustering-then-mapping" philosophy to solve non-linear classification problems.

Network Structure

A CPN consists of two main layers:

- **Kohonen Layer (Competitive Layer)**

  The Kohonen layer is trained in an unsupervised way. Its role is to detect similarities in the input data. Each neuron in this layer represents a prototype of the input space.

  For each input sample, the distance between the input and all Kohonen neurons is computed. The neuron with the **minimum distance** is selected as the winner (Winner-Take-All):

$$winner = \arg\min_{i} \| x - w_i \|$$

  Only the winning neuron updates its weights to move closer to the input:

$$w_i^{new} = w_i^{old} + \alpha(x - w_i^{old})$$

- **Grossberg Layer**

  The Grossberg layer is trained in a supervised manner. It maps the winning Kohonen neuron to the desired output class (e.g., C1 or C2).

  The weights of this layer are updated using:

$$v_{ij}^{new} = v_{ij}^{old} + \beta(t_j - v_{ij}^{old})$$

  where:

  - t_j is the target output (class label)
  - β is the learning rate of the Grossberg layer

Distance Calculation:
To identify the winning neuron, the Euclidean distance between the input vector $x$and each Kohonen weight vector $w_j$is computed as:

$$d_j = \sqrt{\sum_{i=1}^{n} (x_i - w_{ij})^2}$$

The neuron with the smallest distance is selected as the winner, since it represents the prototype that is most similar to the input sample.

Project Objectives and Tasks :

The main objectives of the project are summarized as follows:

- **Task 1 – CPN Design and Investigation:**
  Design and implement a Counter-Propagation Network (CPN) to solve the same problem introduced in last project . Then, study how the performance of the CPN changes when varying the number of neurons in the Kohonen layer.
- **Task 2 – Model Comparison:**
  Compare the performance of the Counter-Propagation Network with the previously implemented Back-Propagation Network (BPN).
- **Task 3 – Discussion:**
  Analyze and discuss the advantages and limitations of using a CPN for this problem.

Key Tasks Performed

1. Pattern Selection: using the data set was generated in the first project.

2. Evaluation: Assessing the network's classification performance using metrics such as Accuracy, Confusion Matrix.

3. Visualization: Plotting the Decision Boundaries to observe the regions created .

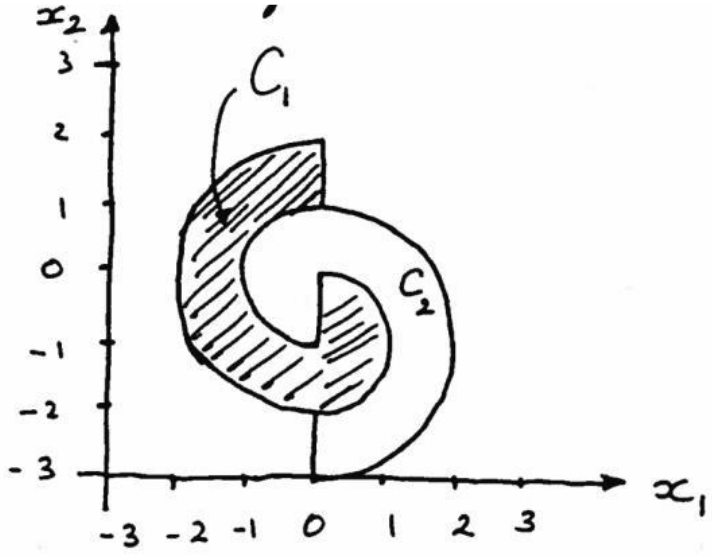
4. Parametric Study: Conducting a detailed investigation into how specific CPN parameters affect performance :

- Number of neurons in the Kohonen layer:
- Learning Rate Optimization (α) and (β)
- random weight Initialization Impact vs. sampling
- Effect of feature scaling
- Comparative Analysis Back-propagation (BPN) model from project #1
- Model Scalability with different Datasets size

Methodology

Dataset Selection:

The data set which was conducted here is the dataset from last project. Which I used a rejection sampling method to generate the points. And I wrote a script to pick random (x1, x2) points between -3 and 3. For each point, I checked its distance from two different centers to see if it fell into the right regions based on the following logic:



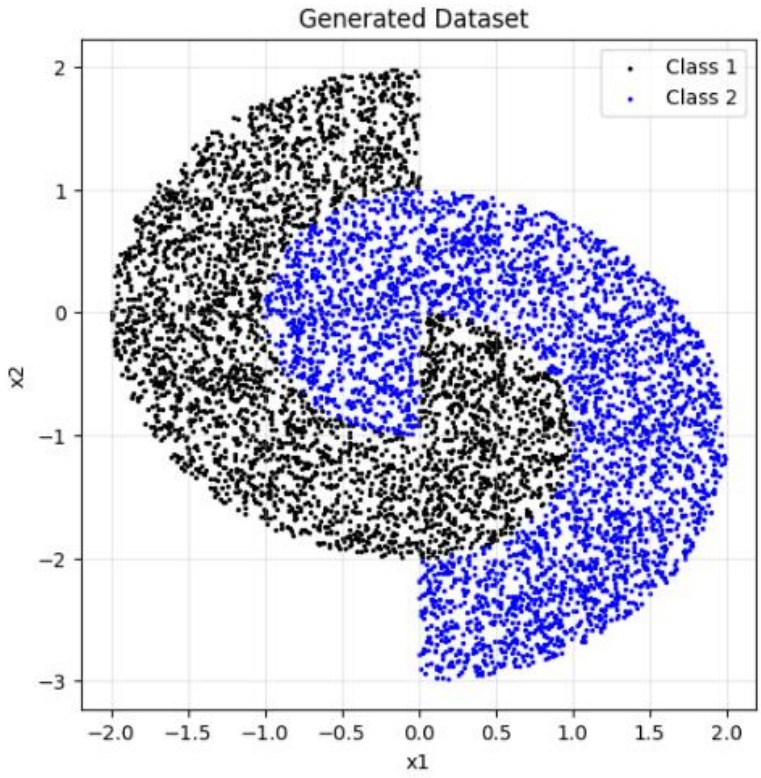
Class C1 :

$$C1 \ :\{ (1 \le x_1^2 + x_2^2 \le 4) \ and \ x_1 \le 0 \} \cup \{(x_1^2 + (x_2 + 1)^2 \le 1) and \ x_1 > 0\}$$

Class C2 :

$$C2 \ : \{(x_1^2 + x_2^2 < 1) \ and \ x_1 \le 0 \} \cup \{(1 \le x_1^2 + (x_2 + 1)^2 \le 4) \ and \ x_1 > 0\}$$

I repeated this sampling process until I collected a total of 7,000 valid samples (approximately 3,500 points per class). This ensures that the dataset is balanced and provides enough information for the neural network to learn the non-linear boundaries. The generated data was then visualized to verify it matches the problem set and saved into a CSV file for the training phase.



Data Preprocessing and Feature Scaling

After generating the raw dataset, since the data generation process utilized random sampling within a loop, the resulting dataset was naturally shuffled, ensuring a random distribution of classes.

- **Data Splitting Strategy:**

To ensure a rigorous evaluation and prevent **data leakage**, the splitting was performed before any normalization steps. The data was partitioned into three subsets: 10

- **Training Set (70%):** Used for weight updates and model learning.
- **Validation Set (10%):** Used for hyperparameter tuning and selecting the best model configuration.
- **Test Set (20%):** Held out until the very end to provide an unbiased evaluation of the final model's performance.

- **Feature Scaling Methodology:**

Standardization was handled carefully to maintain the "unseen" nature of the validation and test data. A StandardScaler was initialized and fitted only on the Training Set to calculate the mean and standard deviation. This process transforms the data to have a mean of 0 and a standard deviation of 1. These training-derived parameters were then used to transform the validation and test sets. This guarantees that the model remains completely unaware of the statistical distribution of the test data during the training phase.

Network Architecture: Counter-Propagation Network (CPN)

The implemented model is a **Counter-Propagation Network**, which acts as a hybrid neural system combining an unsupervised competitive layer with a supervised output layer.

1. Structural Components

- **Input Layer:** Consists of 2 input neurons.
- **Kohonen (Competitive) Layer:** Contains N_k neurons .
- **Grossberg (Interpolative) Layer:** Consists of 2 outputs neurons representing the class labels.

2. Learning Mechanism

- **Winner-Take-All (WTA):** For every input pattern, the network identifies a "Winner" neuron in the Kohonen layer—the one with the minimum Euclidean distance.

- **Hybrid Update Rule:**
    - The Kohonen weights (W_k) are updated using competitive learning to move the centroid closer to the input data. The weights are initialized between **0 and 1** to place neurons near the center of the data. This reduces the chance of dead neurons, which are neurons that never win and therefore never learn.
    - The Grossberg weights (W_g) are simultaneously updated using a supervised delta-rule to associate the winning cluster with the target class. These weights are initialized between **0 and 1** because the target labels are binary.

- **Learning Rate Decay:** To ensure convergence and stability, both the Kohonen rate (α) and Grossberg rate (β) are linearly decreased over 150 epochs. This allows faster learning in the early stages and more fine-tuned updates toward the end of training.

Hyperparameter Tuning Strategy and comparison

To identify the optimal configuration for the Counter-Propagation Network (CPN), a systematic five-phase optimization approach was implemented:

- **Phase 1: Kohonen Layer Sensitivity Analysis:** A parametric study was conducted to determine the impact of the number of neurons in the competitive (Kohonen) layer.
- **Phase 2: Dual Learning Rate Tuning ($\boldsymbol{\alpha}$ & $\boldsymbol{\beta}$):** Optimization of the unsupervised learning rate ($\boldsymbol{\alpha}$) for the Kohonen layer and the supervised rate ($\boldsymbol{\beta}$) for the Grossberg layer.
- **Phase 3: Weight Initialization Impact:** A comparative study between **Random Initialization** and **Data Sampling**.
- **Phase 4: Feature scaling impact:** This evaluated the effects of feature scaling on data.
- **Phase 4: Comparative Evaluation (CPN vs. BPN):** Integration of the best-performing hyperparameters to train the final CPN. The model was then compared against the Back-propagation (BPN) results from project #1 in terms of accuracy, training speed, and boundary smoothness.
- **Phase 6: Scalability & Robustness Study:** Evaluating the CPN's performance on larger datasets (3.5k and 15k samples). This confirmed whether the competitive architecture maintains its accuracy and efficiency as data density increases.

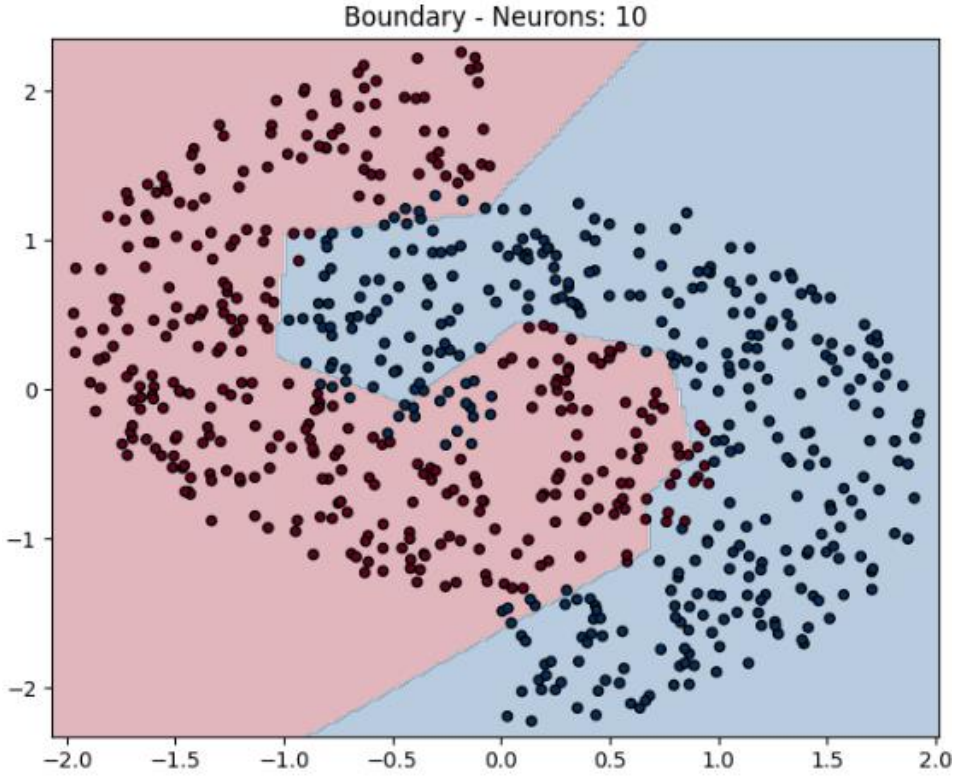
Results & Discussion (Parametric Study)

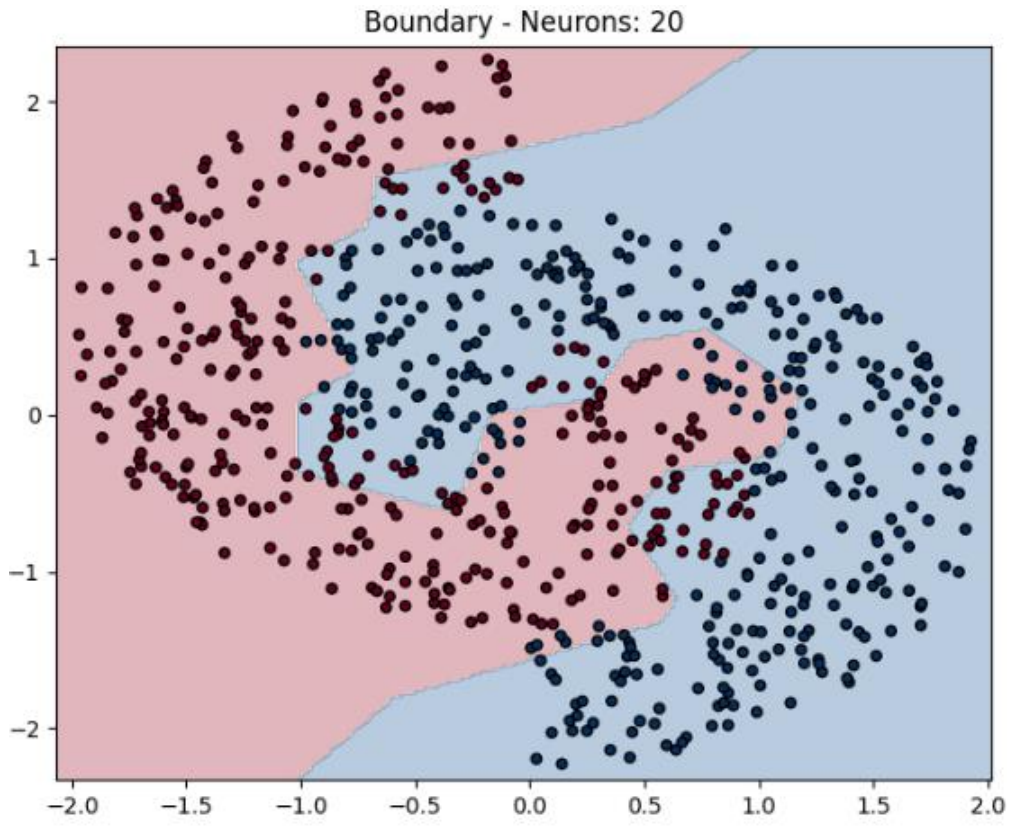
Phase 1: Kohonen Layer Sensitivity Analysis:

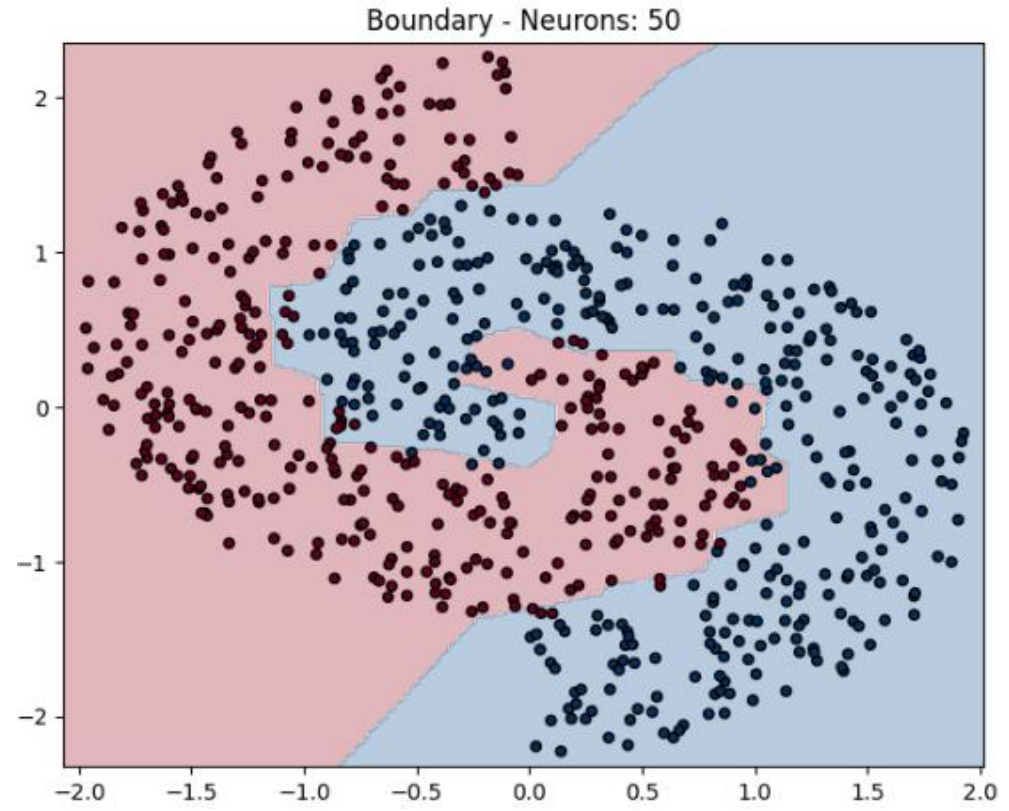
This section analyzes how the number of neurons in the Kohonen (competitive) layer, denoted as N_k, affects the performance of the CPN.

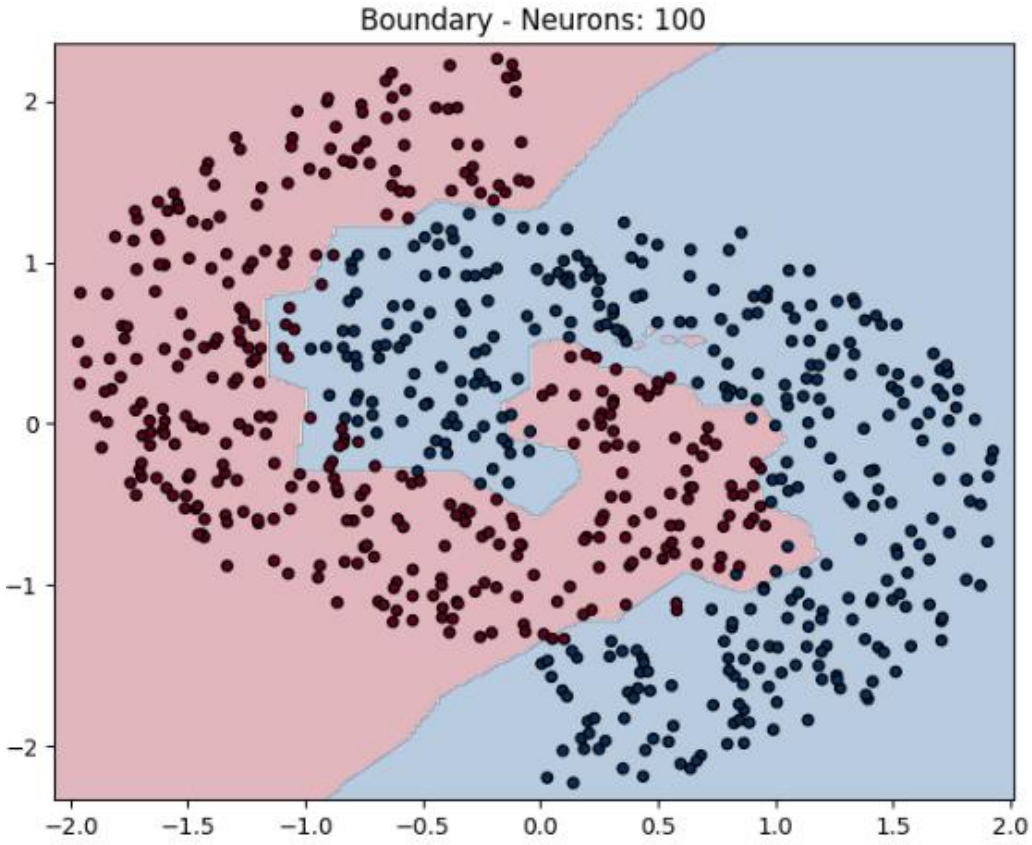
Investigation: Varying $N_k \in$**{10, 50, 100, 200, 500}.**

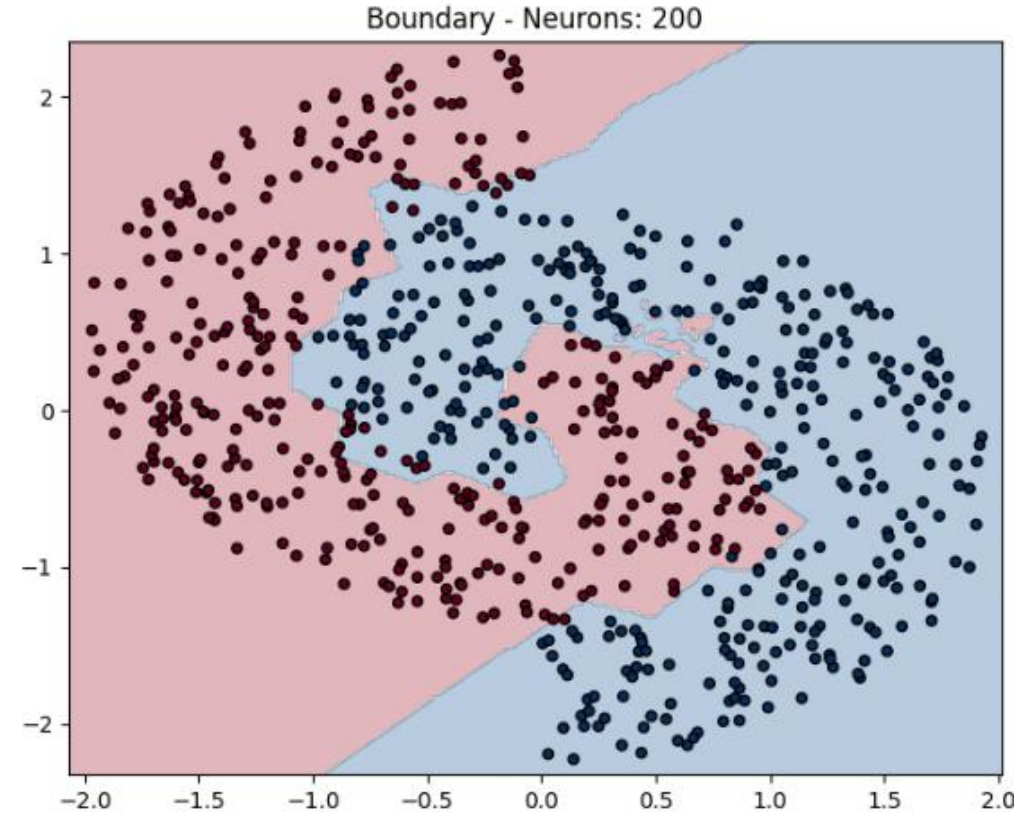
Fix Parameters and hyperparameters:	**Values:**
Number of Hidden Layers	1
Total Data Size	7,000
Learning rate α	0.01
Learning rate β	0.01
Weight Initialization	Random
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization

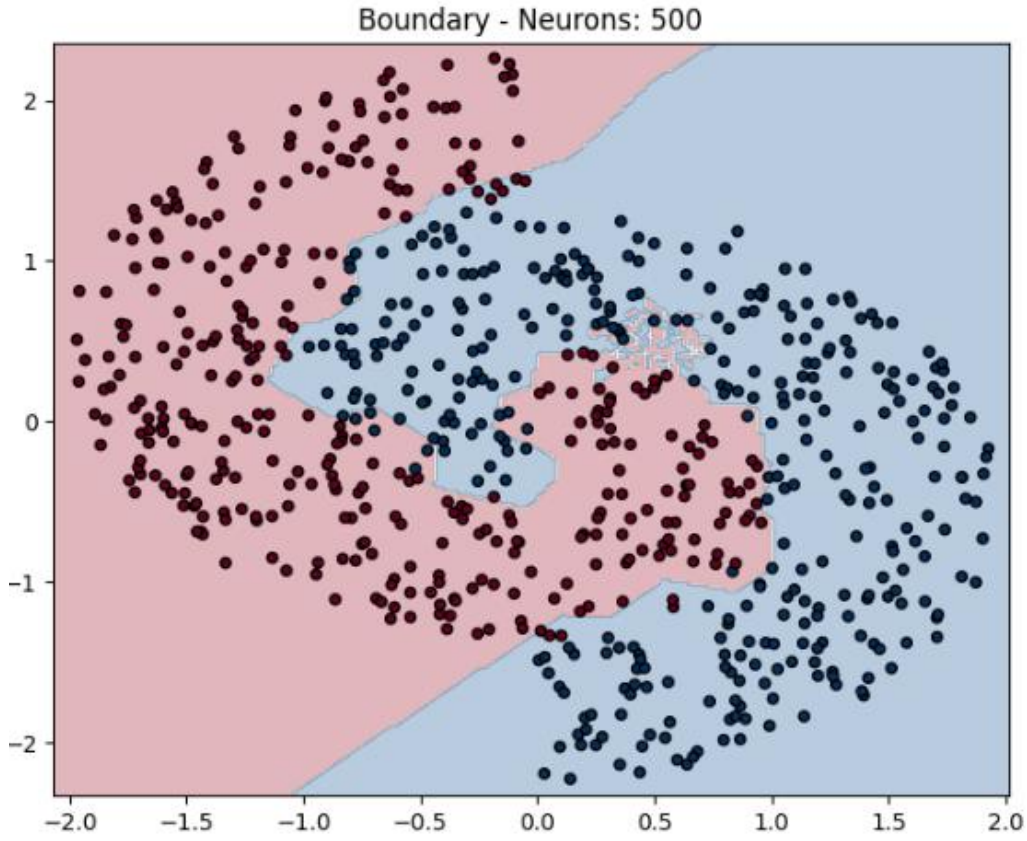


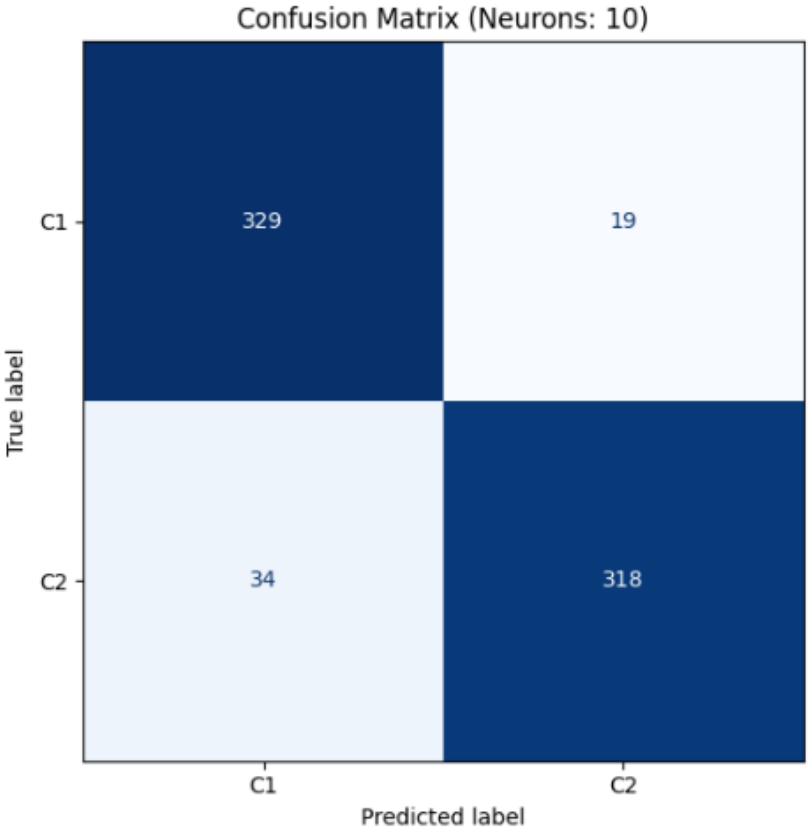

Boundary - Neurons: 20
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


Boundary - Neurons: 50
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

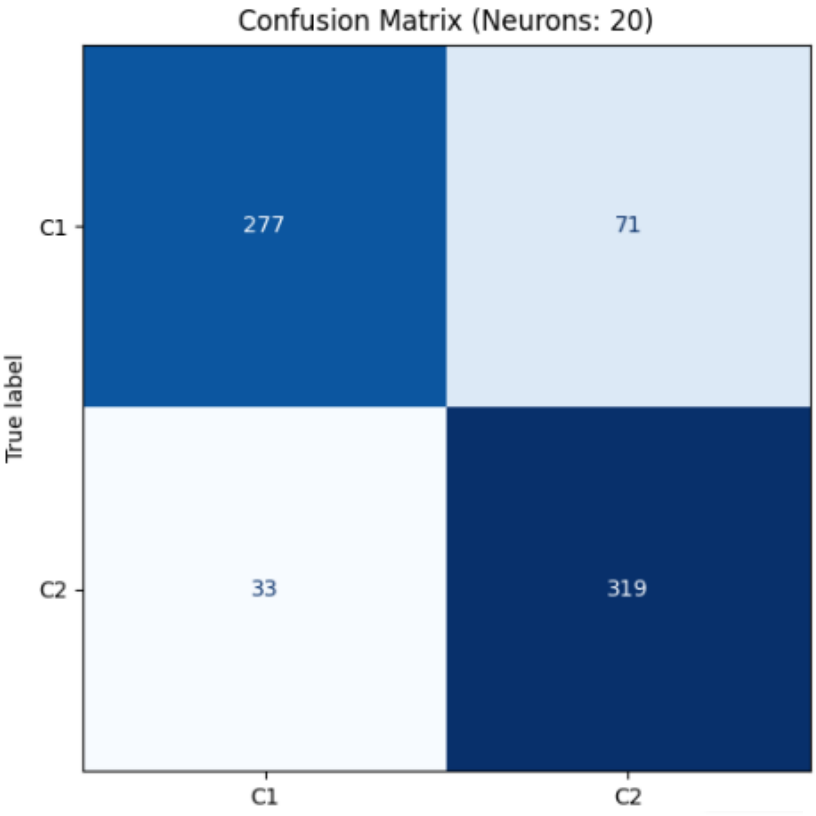

Boundary - Neurons: 100
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


Boundary - Neurons: 200
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

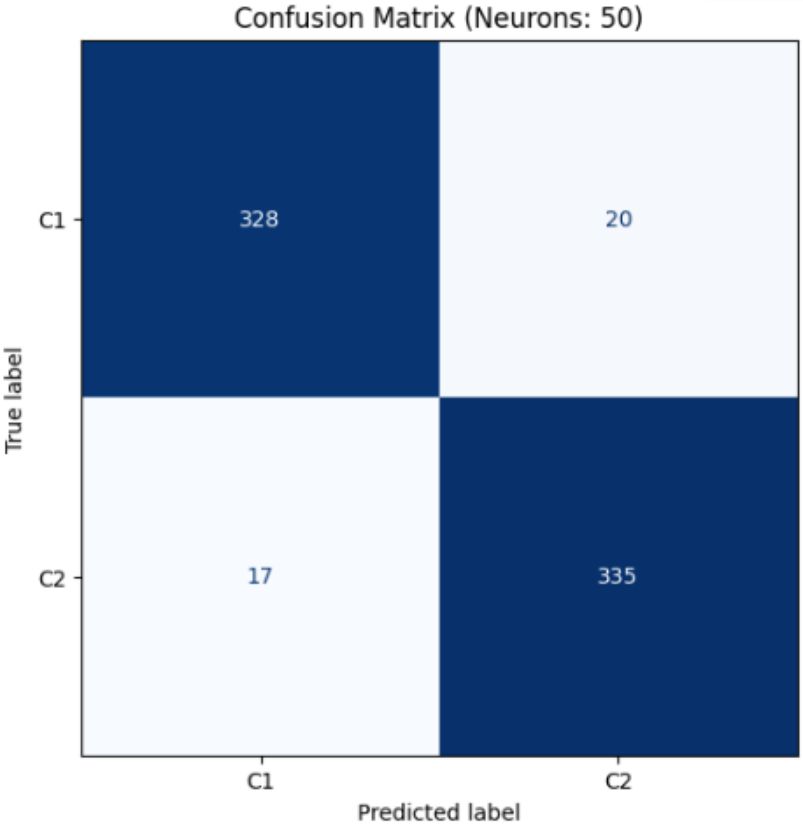




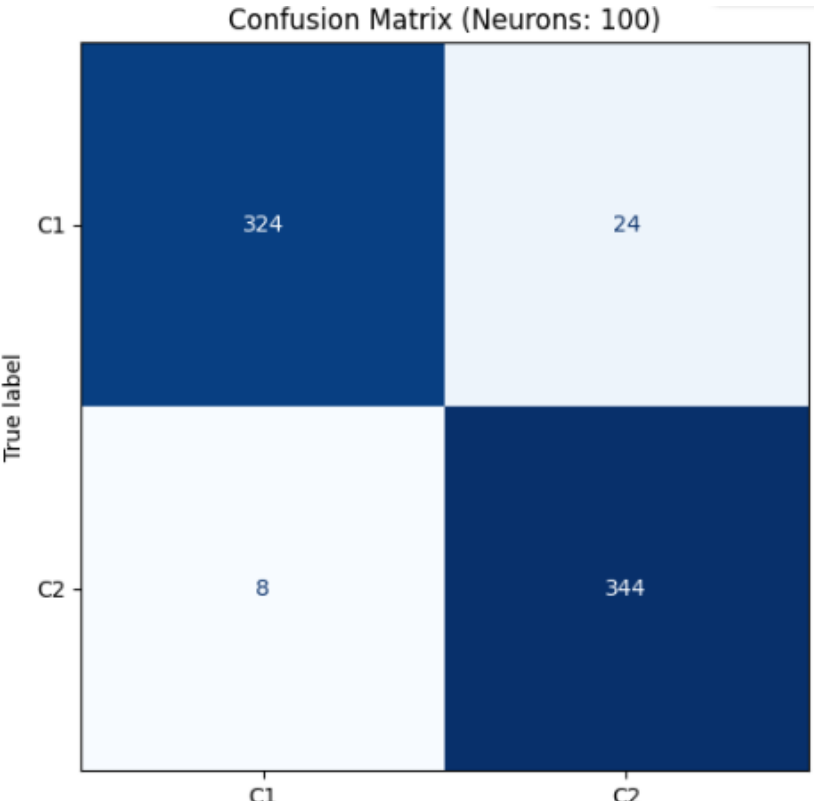

```
Running Experiment: 10 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.91      0.95      0.93       348
          C2       0.94      0.90      0.92       352

    accuracy                           0.92       700
   macro avg       0.92      0.92      0.92       700
weighted avg       0.93      0.92      0.92       700

Final Validation Accuracy: 92.43%
```



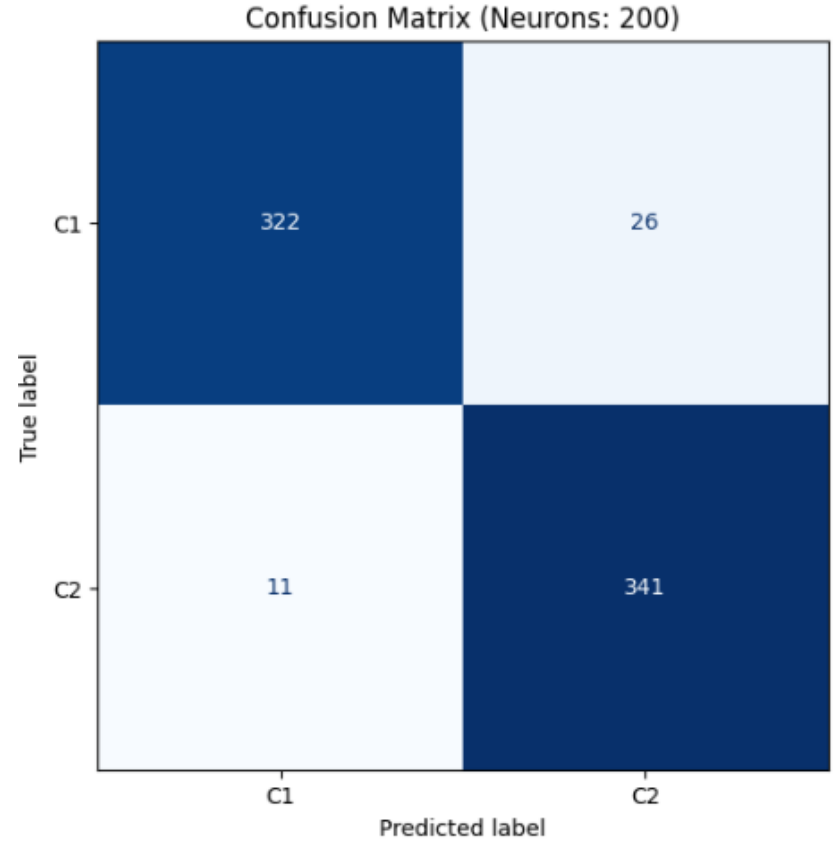

```
Running Experiment: 20 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.89      0.80      0.84       348
          C2       0.82      0.91      0.86       352

    accuracy                           0.85       700
   macro avg       0.86      0.85      0.85       700
weighted avg       0.86      0.85      0.85       700

Final Validation Accuracy: 85.14%
```



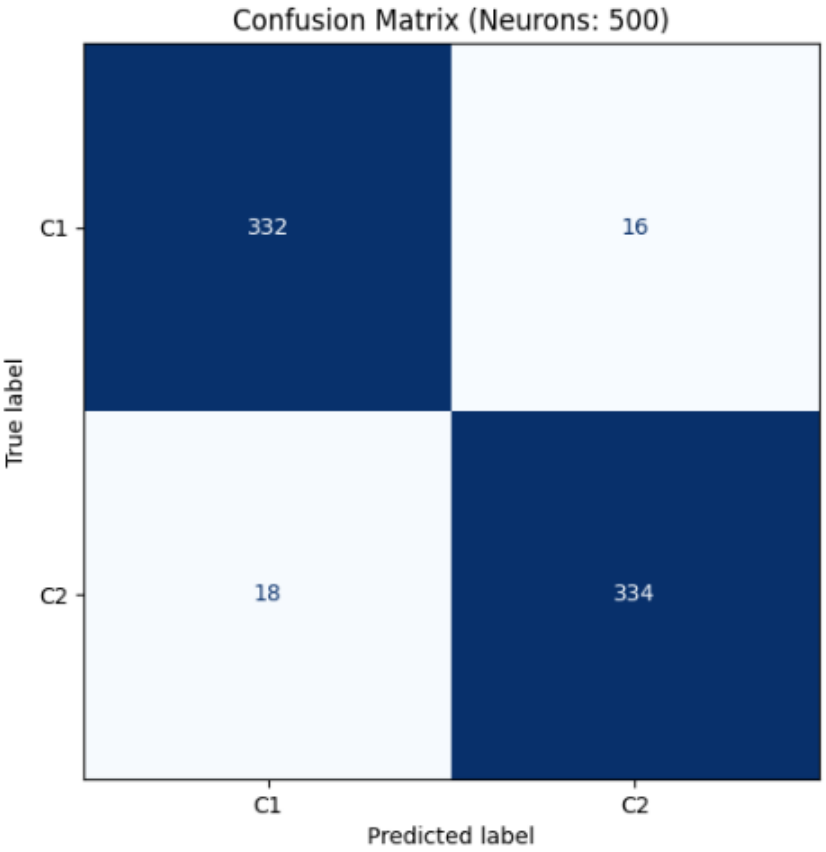

```
Running Experiment: 50 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.95      0.94      0.95       348
          C2       0.94      0.95      0.95       352

    accuracy                           0.95       700
   macro avg       0.95      0.95      0.95       700
weighted avg       0.95      0.95      0.95       700

Final Validation Accuracy: 94.71%
```



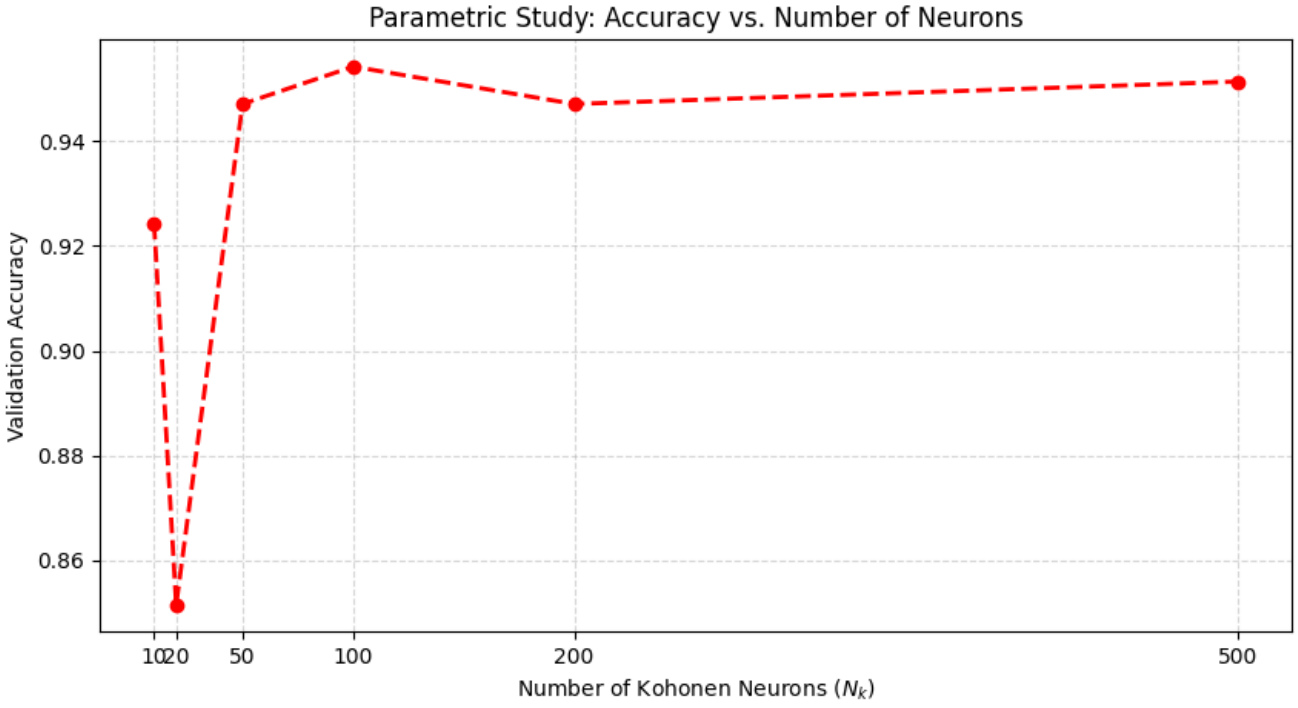

```
Running Experiment: 100 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.98      0.93      0.95       348
          C2       0.93      0.98      0.96       352

    accuracy                           0.95       700
   macro avg       0.96      0.95      0.95       700
weighted avg       0.96      0.95      0.95       700

Final Validation Accuracy: 95.43%
```

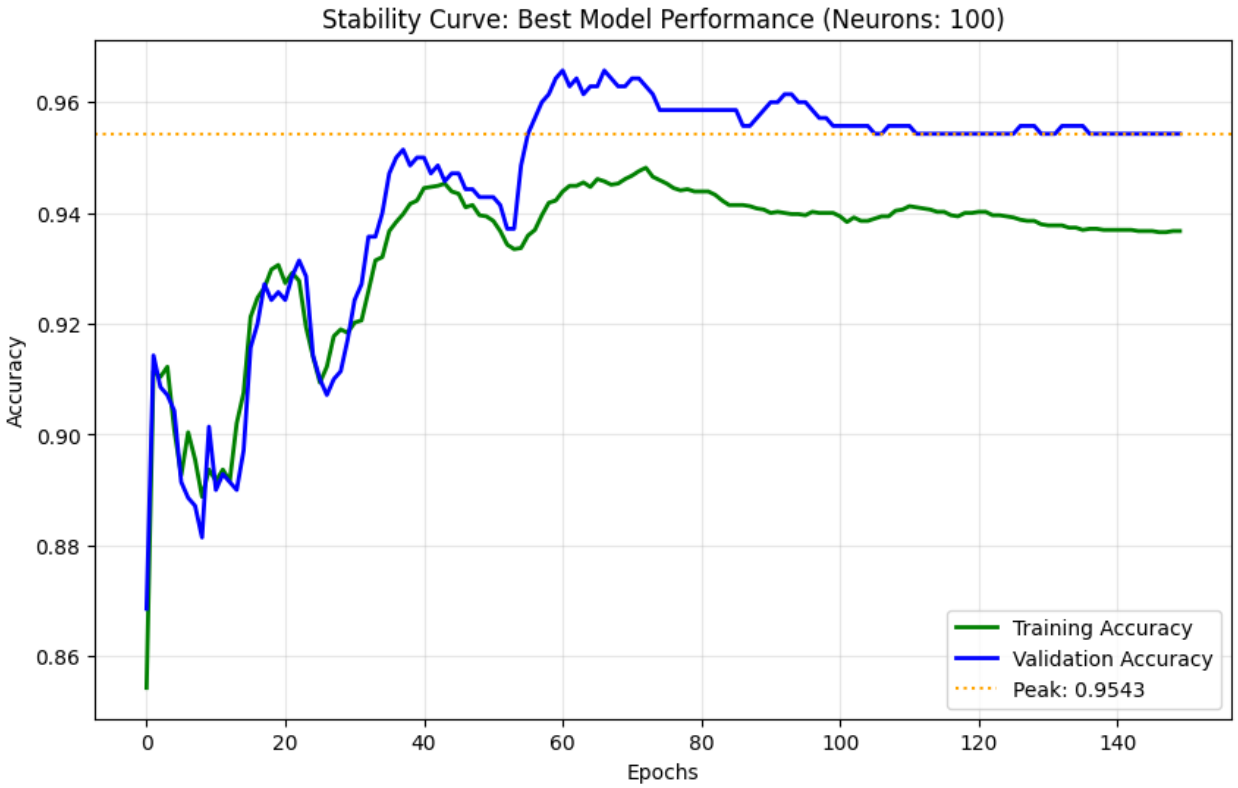


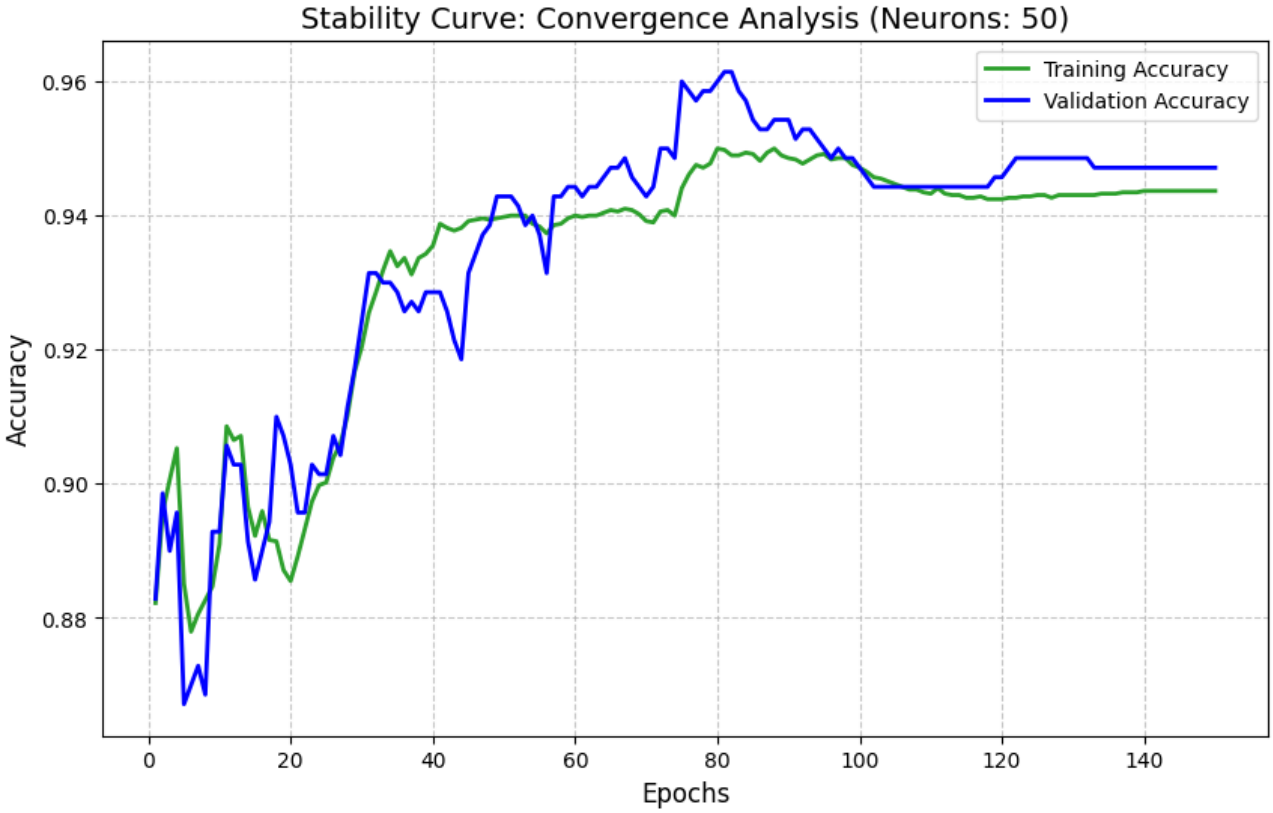

```
Running Experiment: 200 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.97      0.93      0.95       348
          C2       0.93      0.97      0.95       352

    accuracy                           0.95       700
   macro avg       0.95      0.95      0.95       700
weighted avg       0.95      0.95      0.95       700

Final Validation Accuracy: 94.71%
```




```
Running Experiment: 500 Kohonen Neurons
              precision    recall  f1-score   support

          C1       0.95      0.95      0.95       348
          C2       0.95      0.95      0.95       352

    accuracy                           0.95       700
   macro avg       0.95      0.95      0.95       700
weighted avg       0.95      0.95      0.95       700

Final Validation Accuracy: 95.14%
```




Stability Curve: Best Model Performance (Neurons: 100)
Accuracy
Epochs
Training Accuracy
Validation Accuracy
Peak: 0.9543


Stability Curve: Convergence Analysis (Neurons: 50)
Accuracy
Epochs
Training Accuracy
Validation Accuracy

When using **10 neurons**, the model achieved a reasonable accuracy of **92.43%,** but the decision boundary was very blocky, which indicates underfitting. At **20 neurons**, the accuracy dropped sharply to **85.14%.** This drop suggests the number of neurons is still too small to properly cover the circular structure of the data.

From 50 neurons onward, the model performance improved significantly and became much more stable. The **50-neuron model** reached **94.71% accuracy** and produced a smooth and well-balanced decision boundary and learns more smoothly, which means it generalizes better. However, its training curve shows some instability near the end of training, including a sudden jump in accuracy around epoch 120. This suggests that with fewer neurons, the network may struggle to keep a stable partition of the input space.

100-neuron model reaches the highest validation accuracy **(95.43%)** and shows very stable convergence. After around epoch 110, the accuracy curve becomes completely flat, indicating that the network has reached a steady and reliable solution. The main drawback is that the decision boundary becomes more jagged, means the model fits the data more tightly.

When the number of neurons is increased further to **500**, the accuracy slightly drops to **95.14%**, showing that the model has reached a saturation point.

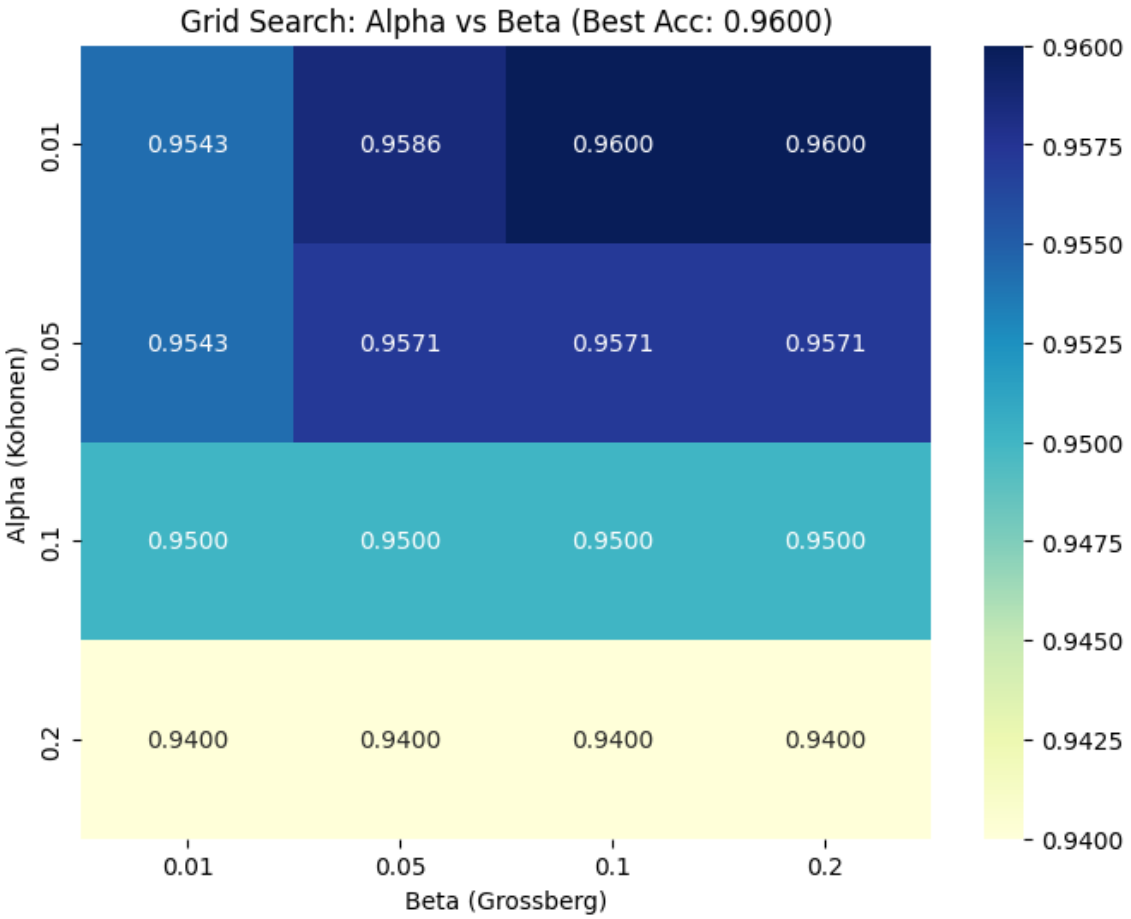
In summary, while 50 neurons provide the smoother boundaries, the **100-neuron** Kohonen layer is the **most practical and reliable** choice due to its **higher accuracy**, stable learning behavior, and more complexity.

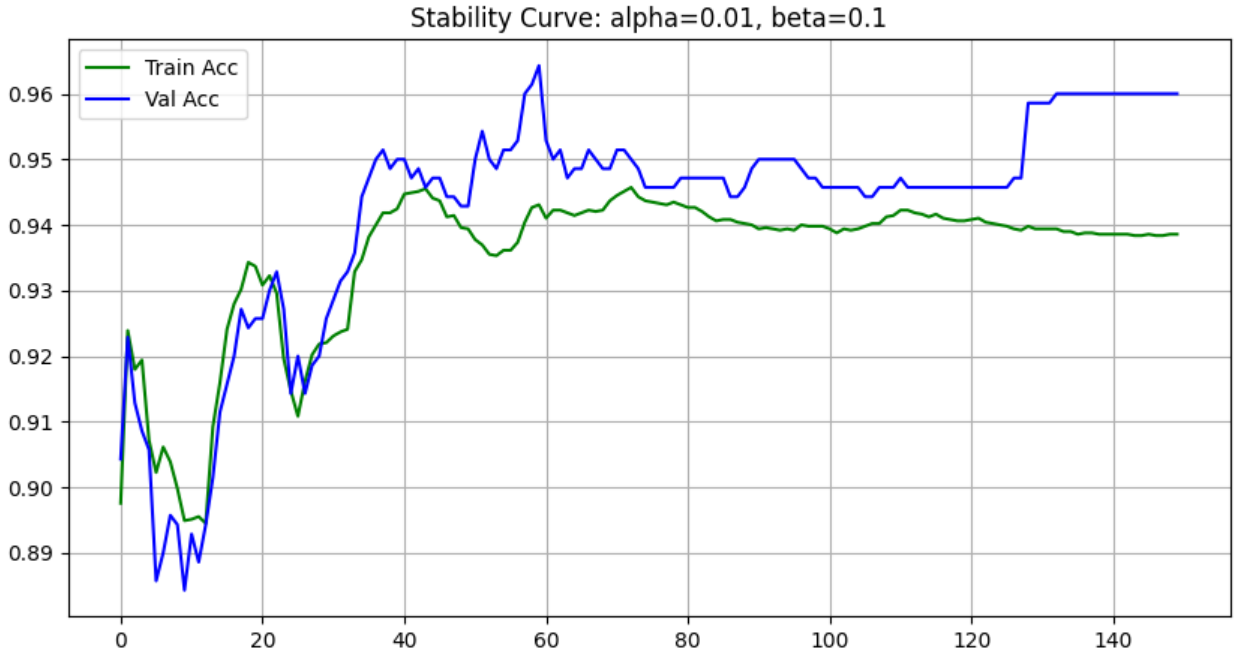
Phase 2: Dual Learning Rate Tuning (α & β):

This section analyzes how the Learning Rates of $\boldsymbol{\alpha}$ **&** $\boldsymbol{\beta}$ affect the performance of the CPN using with a Grid Search.

- **Investigation:** $\alpha \in \{ 0.01, 0.05, 0.1, 0.2 \}$, $\beta \in \{ 0.01, 0.05, 0.1, 0.2 \}$
- **Experiment 1 :**

Fix Parameters and hyperparameters:	Values:
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Total Data Size	7,000
Weight Initialization	Random
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization


Grid Search: Alpha vs Beta (Best Acc: 0.9600)
Alpha (Kohonen)
Beta (Grossberg)
0.01
0.05
0.1
0.2
0.9543
0.9586
0.9600
0.9600
0.9543
0.9571
0.9571
0.9571
0.9500
0.9500
0.9500
0.9500
0.9400
0.9400
0.9400
0.9400
0.9600
0.9575
0.9550
0.9525
0.9500
0.9475
0.9450
0.9425
0.9400


Stability Curve: alpha=0.01, beta=0.1
Train Acc
Val Acc
0.96
0.95
0.94
0.93
0.92
0.91
0.90
0.89
0
20
40
60
80
100
120
140

The **highest validation accuracy (96.00%)** occurred at $\boldsymbol{\alpha}$ **= 0.01** with $\boldsymbol{\beta}$ **= 0.1 (or 0.2)**. As $\boldsymbol{\alpha}$ increased, the accuracy gradually dropped, reaching about **94%** when $\boldsymbol{\alpha}$ **= 0.2**. This clearly shows that the model is much more sensitive to the Kohonen learning rate than to the Grossberg learning rate.

A small $\boldsymbol{\alpha}$ works better because it allows the Kohonen neurons to move slowly. This careful movement helps the network form accurate clusters. When $\boldsymbol{\alpha}$ is too large, the neurons move too aggressively and may skip over important boundary regions, which reduces accuracy.

On the other hand, changing $\boldsymbol{\beta}$ has much less impact on performance. For a fixed $\boldsymbol{\alpha}$, the accuracy stays almost the same across different $\boldsymbol{\beta}$ values.

The accuracy curve of the best model shows some fluctuations during the early epochs, especially between epochs 0 and 60. This phase represents exploration, where the competitive neurons are still adjusting their positions. After around epoch 130, the curve becomes flat, showing that the model has fully converged.

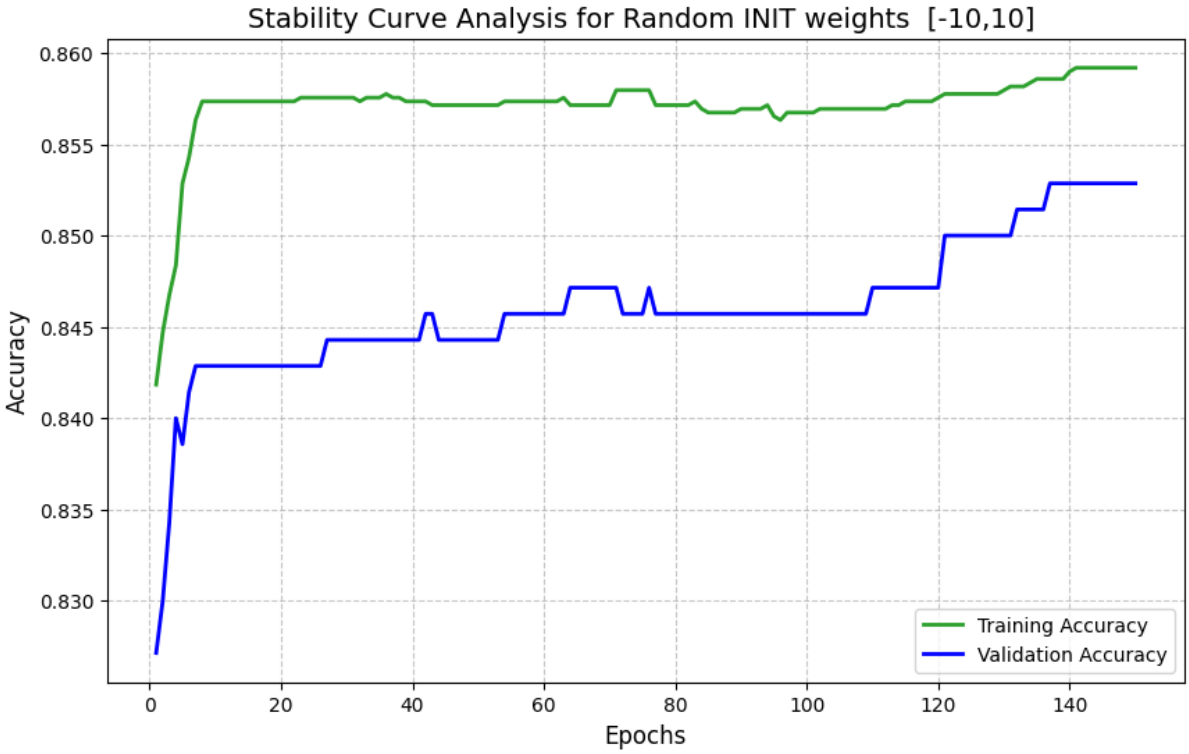
Phase 3. Weight Initialization Strategy

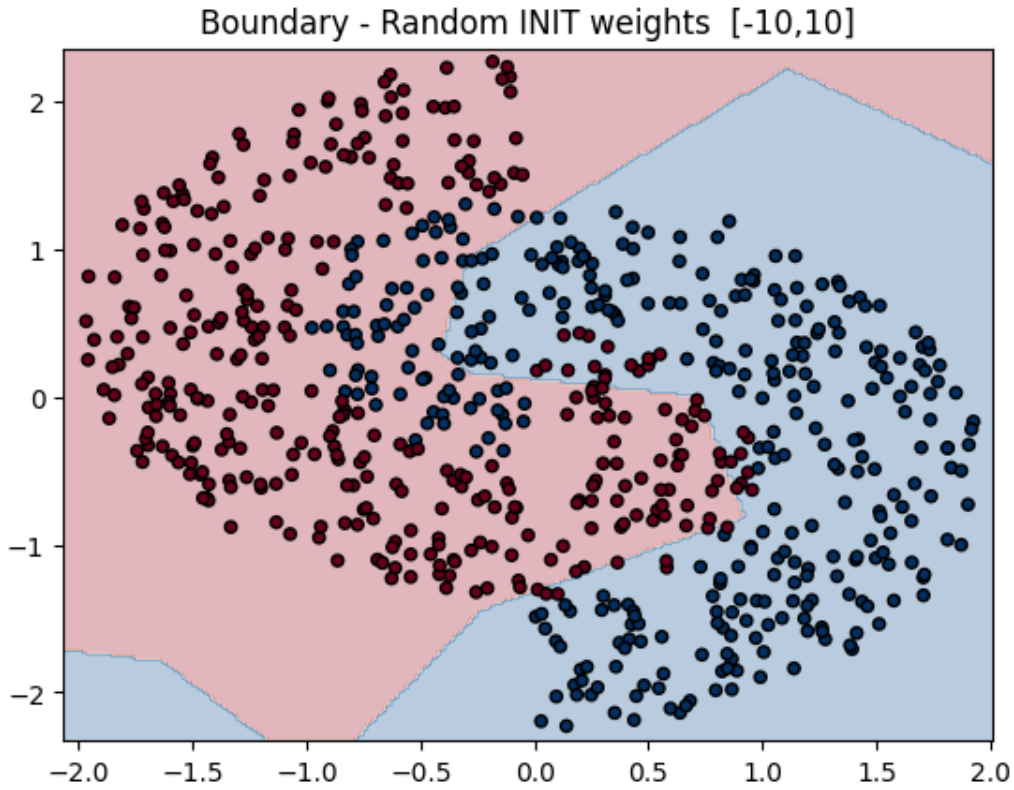
This section analyzes how selecting Initialization weights for Kohonen layer (W_k) affect the performance of the CPN .

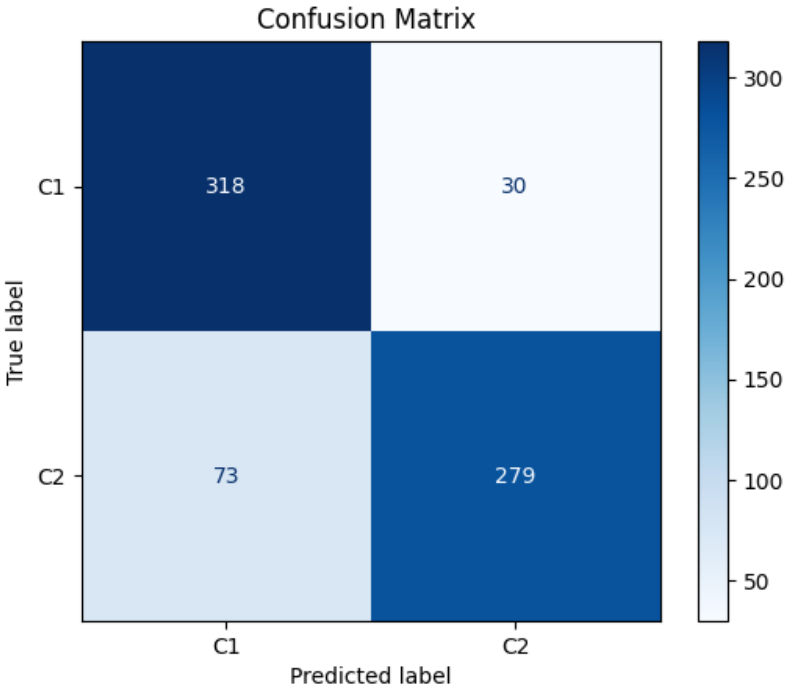
- **Random Initialization:** Weights are initialized between [-10, 10].
- **The Weights Initialization we used:** Weights are randomly initialized between [0, 1].
- **Sampling:** Selecting initial weights directly from the training data points.

Fix Parameters and hyperparameters:	**Values:**
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Total Data Size	7,000
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization
Learning Rate α	0.01
Learning Rate β	0.1

Random Initialization between [-10, 10] :

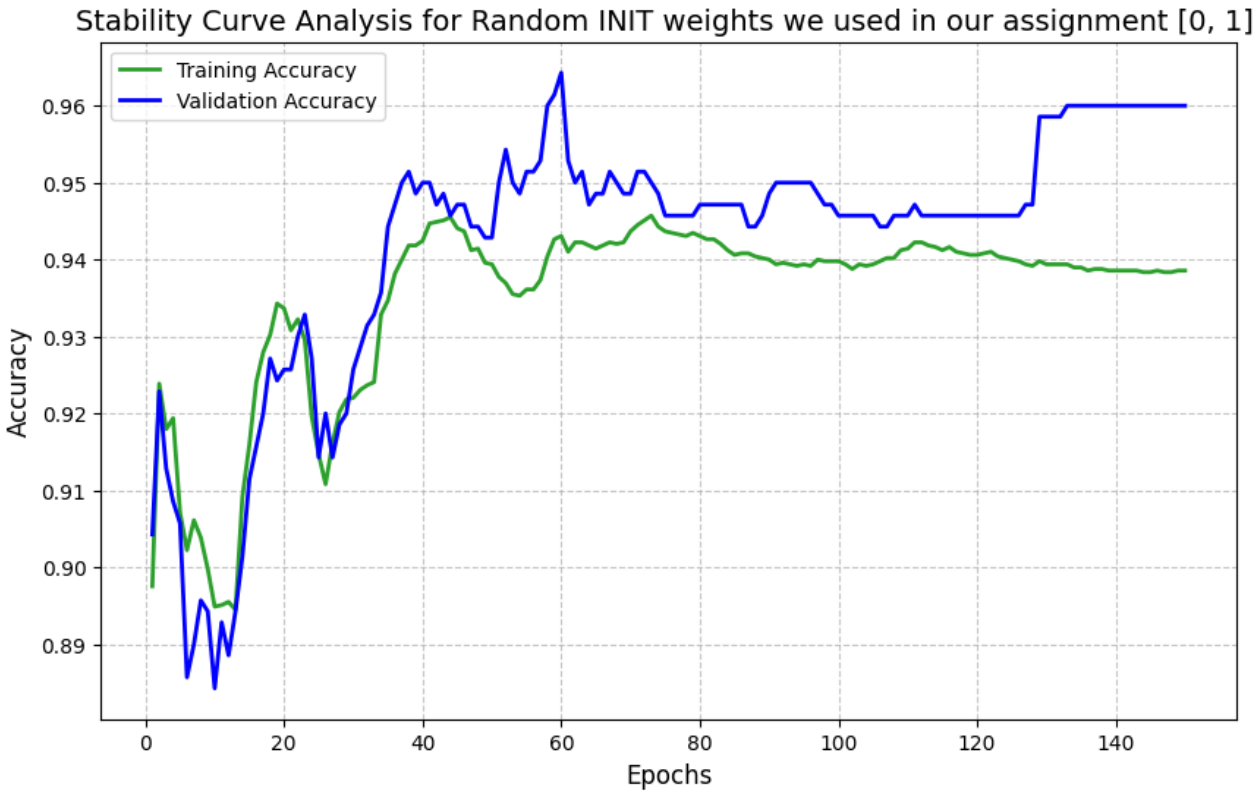


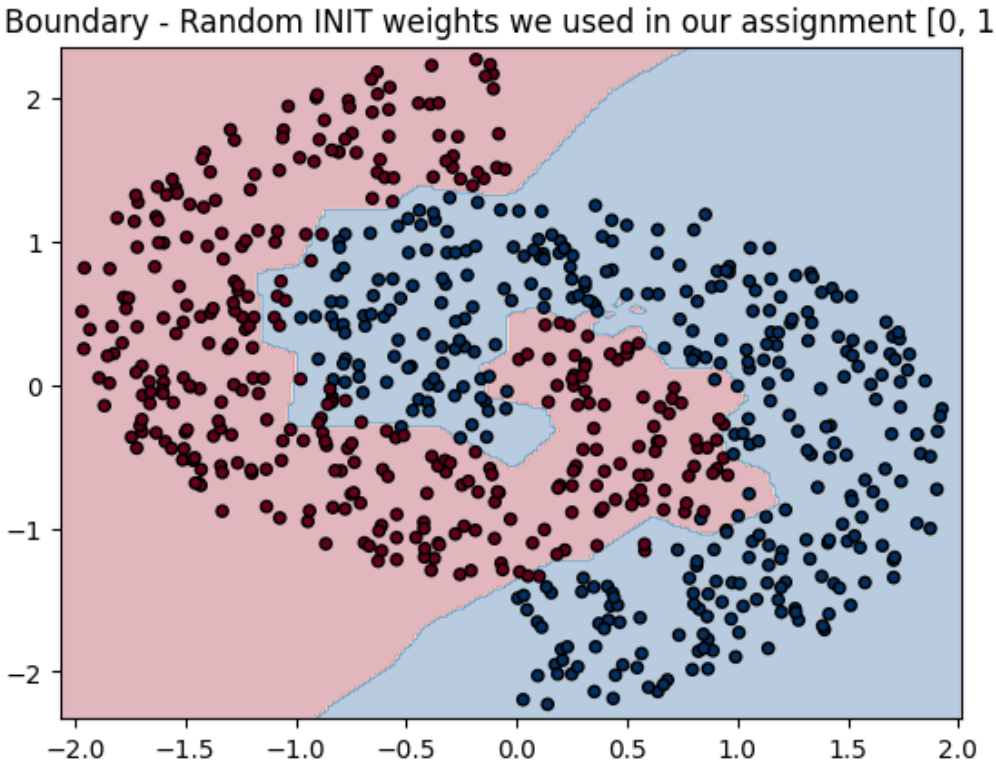


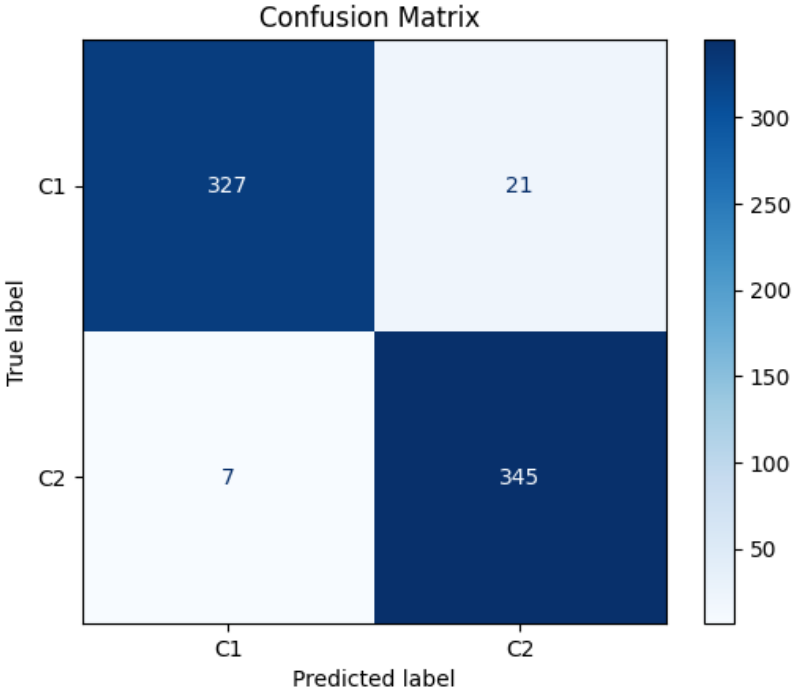



```
--- classification_report
              precision    recall  f1-score   support

          C1       0.81      0.91      0.86       348
          C2       0.90      0.79      0.84       352

    accuracy                           0.85       700
   macro avg       0.86      0.85      0.85       700
weighted avg       0.86      0.85      0.85       700

 Final Validation Accuracy: 85.29%
```

Random Initialization between [0, 1] :

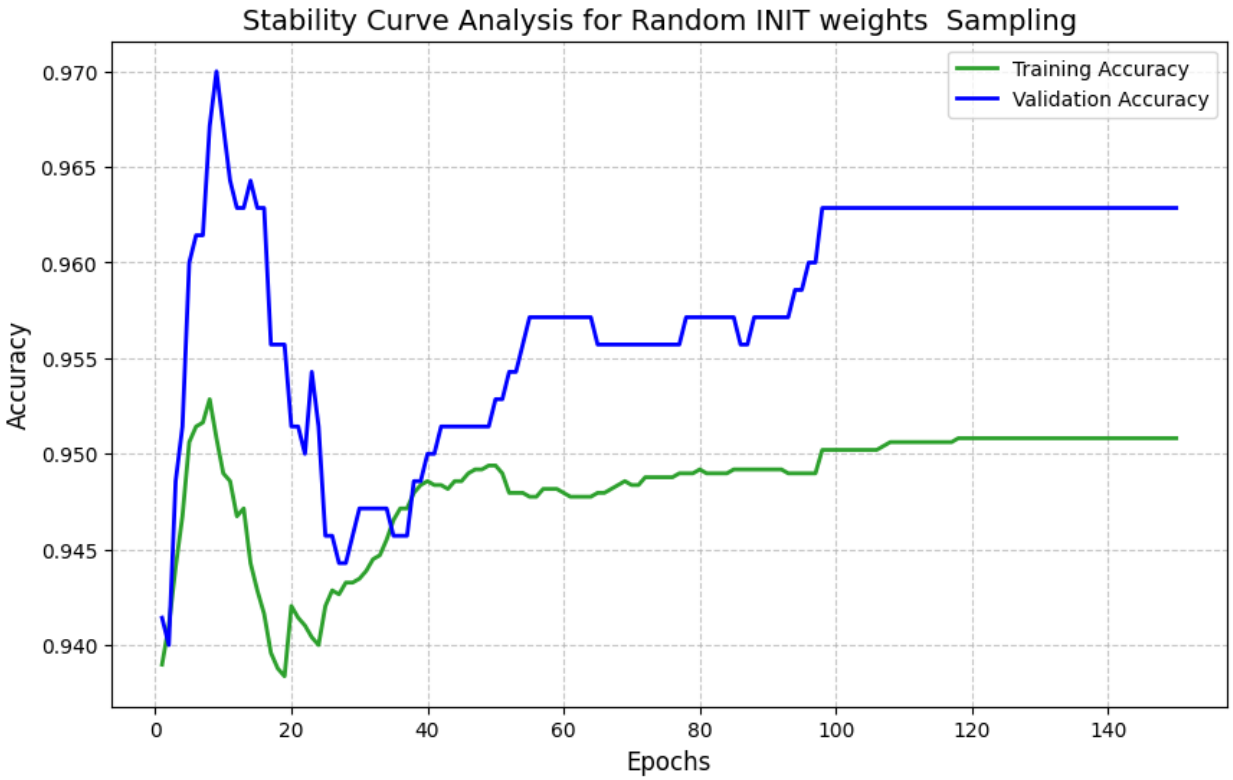


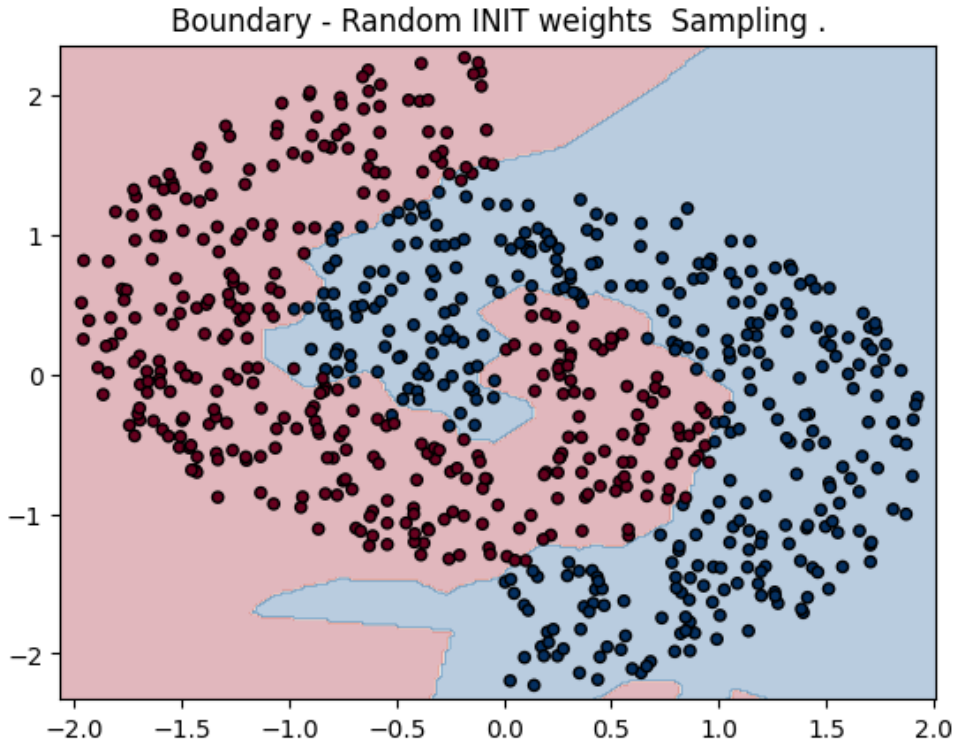


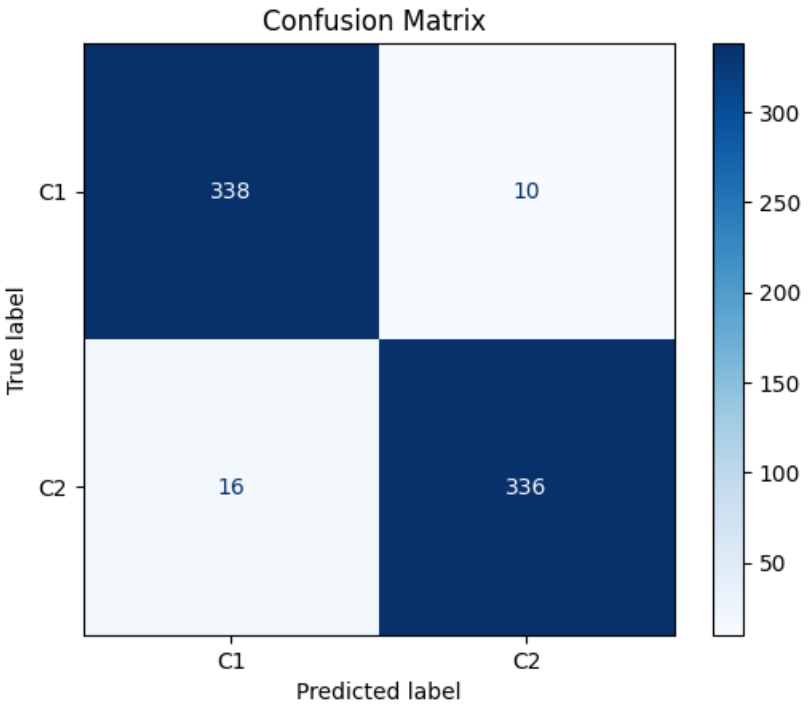



```
--- classification_report
              precision    recall  f1-score   support

          C1       0.98      0.94      0.96       348
          C2       0.94      0.98      0.96       352

    accuracy                           0.96       700
   macro avg       0.96      0.96      0.96       700
weighted avg       0.96      0.96      0.96       700

 Final Validation Accuracy: 96.00%
```

Sampling:


Stability Curve Analysis for Random INIT weights Sampling
Training Accuracy
Validation Accuracy
Accuracy
Epochs
0.970
0.965
0.960
0.955
0.950
0.945
0.940
0
20
40
60
80
100
120
140


Boundary - Random INIT weights Sampling .
2
1
0
−1
−2
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0




```
--- classification_report
              precision    recall  f1-score   support

          C1       0.95      0.97      0.96       348
          C2       0.97      0.95      0.96       352

    accuracy                           0.96       700
   macro avg       0.96      0.96      0.96       700
weighted avg       0.96      0.96      0.96       700

 Final Validation Accuracy: 96.29%
```

With extreme **random initialization (range $[-10, 10]$)**, the network performs very poorly about **85.29%**. Training is unstable, accuracy stays low, and the decision boundary is very rough. Many neurons become dead because they start too far from the data, which leads to a large number of classification errors.

The strategic **random initialization (range $[0, 1]$)** which we used as **base configuration** for our model performs much better about **96.00%.** The model eventually reaches high accuracy, but training is noisy and unstable for many epochs. The decision boundary follows the shape of the circles almost well, though it remains somewhat jagged.

The best results come from **sampling**, where weights are initialized using random points from the training data. This model starts with high accuracy, converges very fast, and remains stable throughout training at **96.29%.** It also produces the smoothest and most accurate decision boundary, with the fewest errors overall.

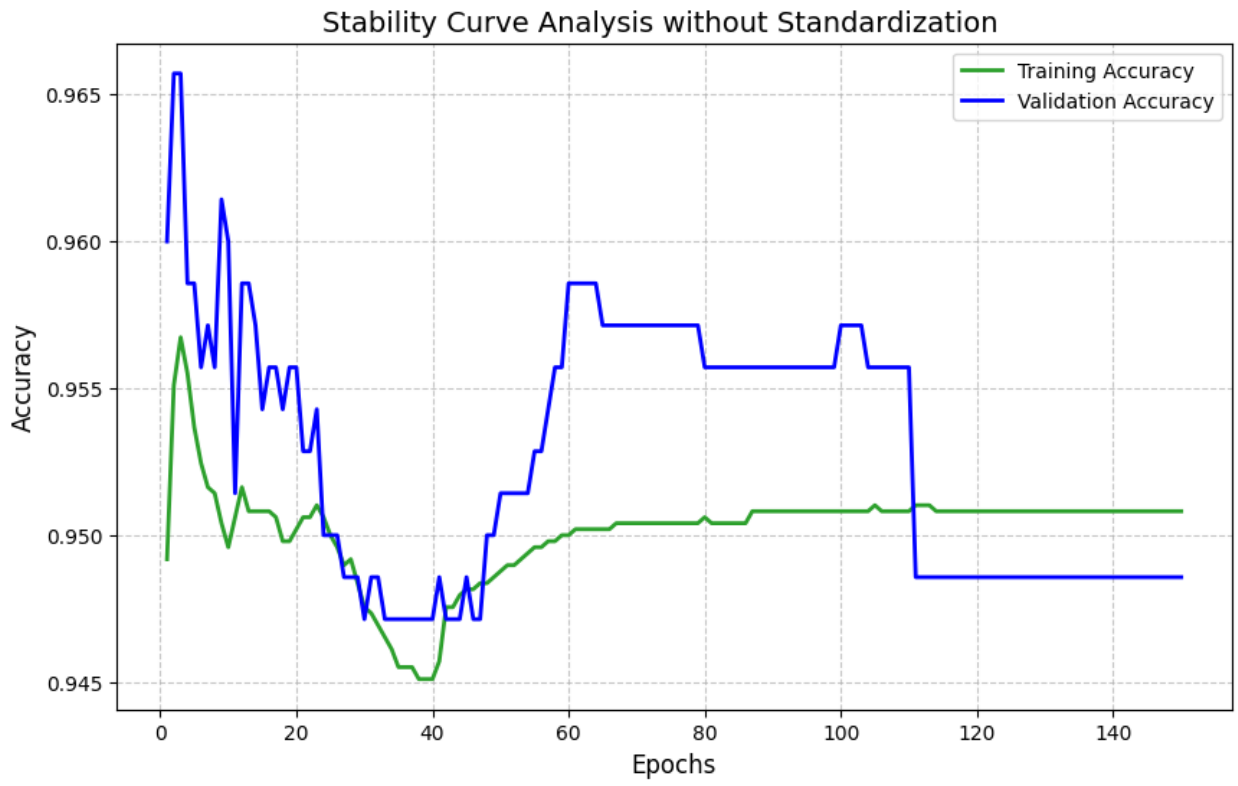
The results show that Sampling is the best initialization method. By initializing the neurons directly from the data points, the network starts closer to the real data structure. This helps avoid the dead neuron problem and allows the model to converge faster with better stability and accuracy.

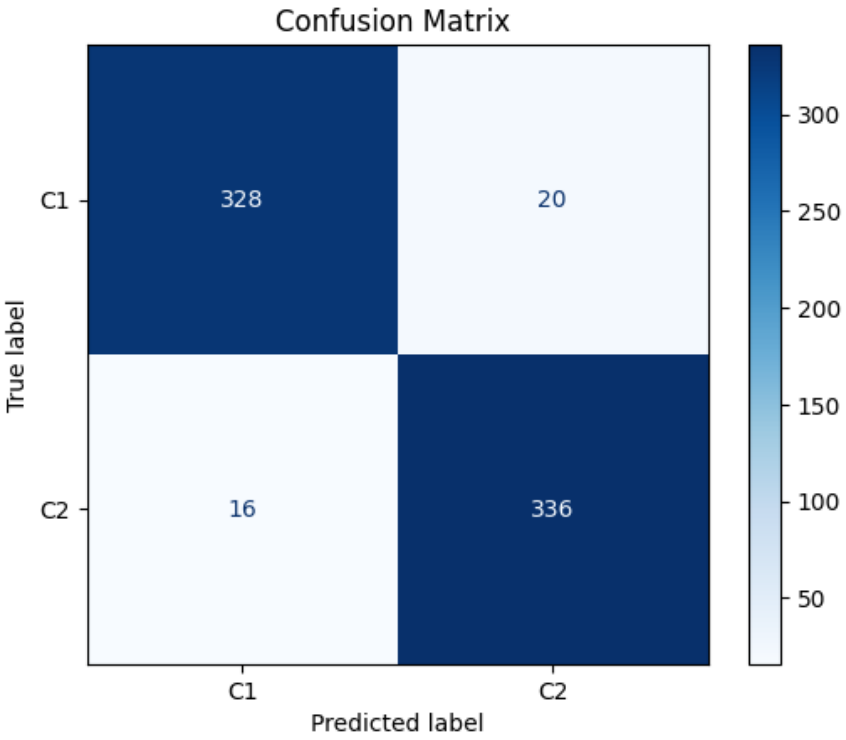
Phase 4: Feature scaling impact :

This phase evaluates how Standardization affects the CPN model's ability to learn the interlocking circles.

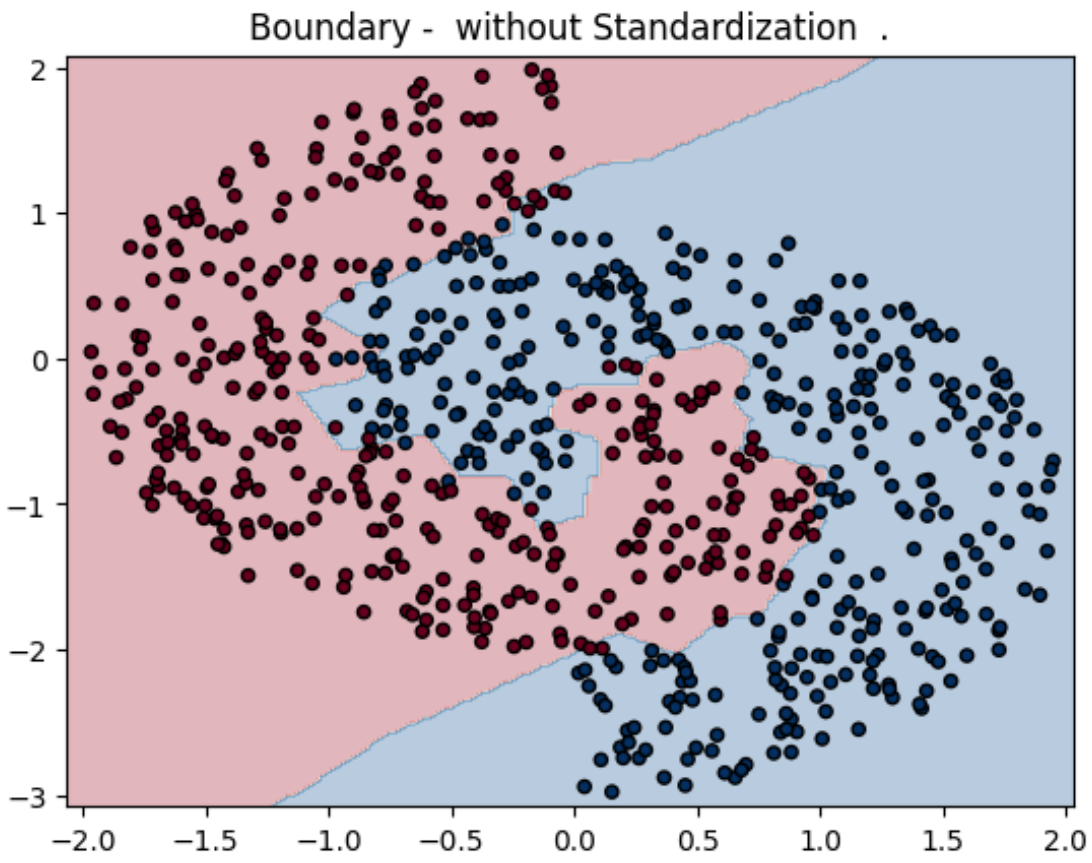
Fix Parameters and hyperparameters CPN:	Values:
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Total Data Size	7,000
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Learning Rate α	0.01
Learning Rate β	0.1
Weight Initialization	competitive sampling

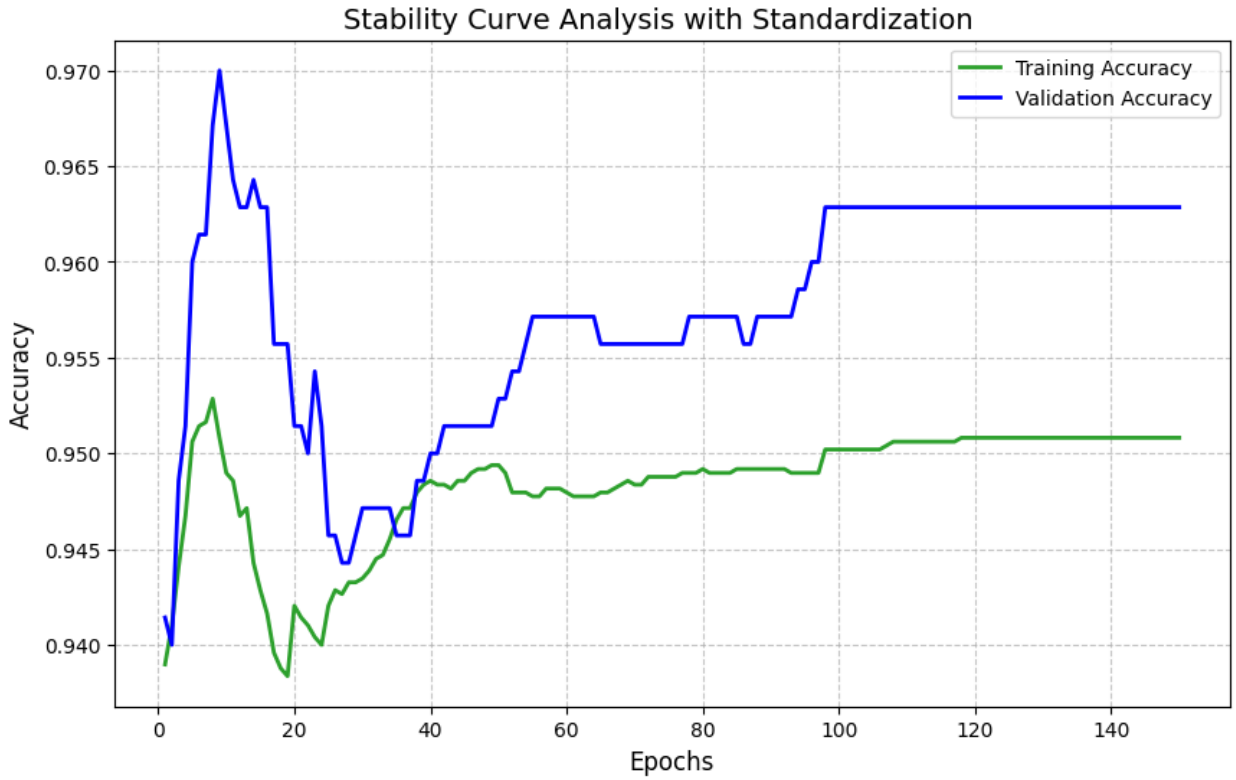
without Standardization:

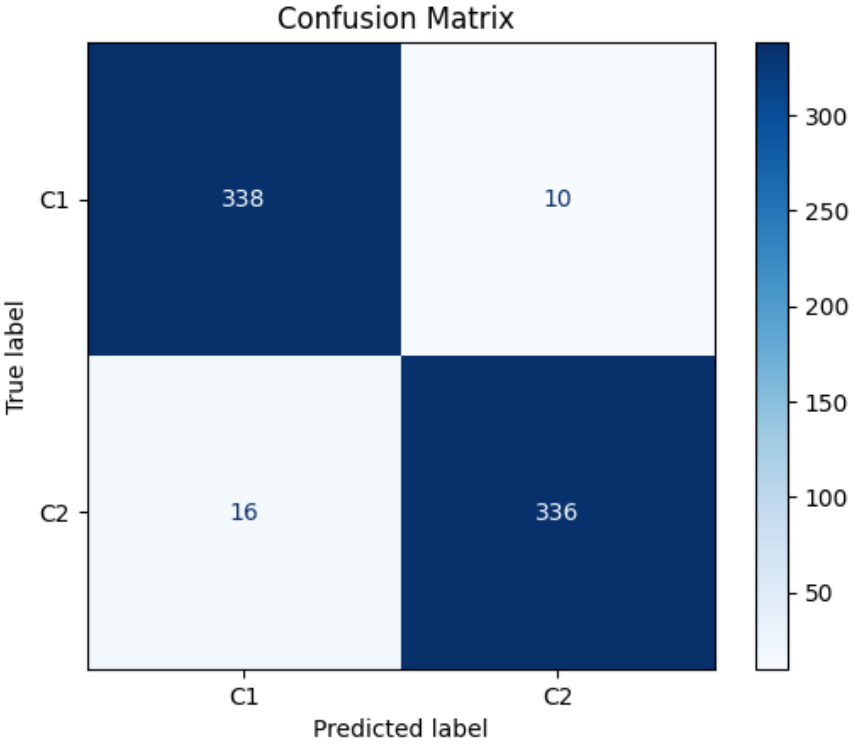




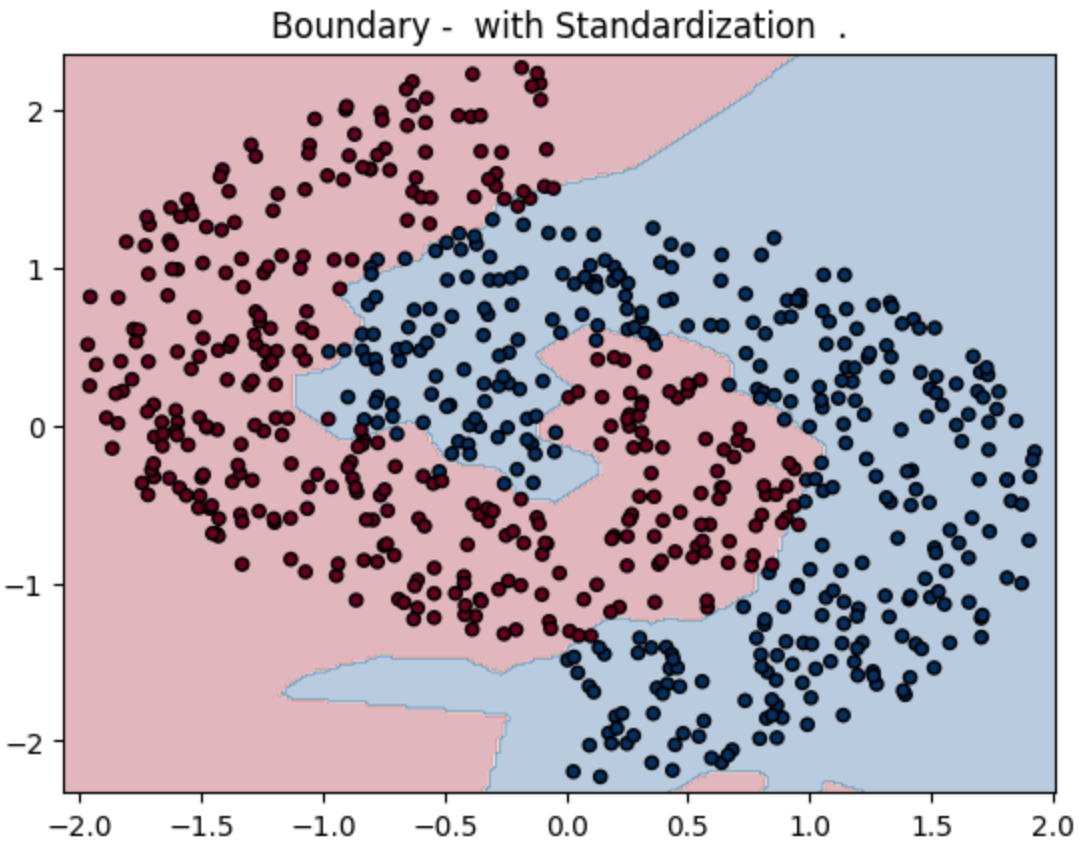

```
--- classification_report
              precision    recall  f1-score   support

          C1       0.95      0.94      0.95       348
          C2       0.94      0.95      0.95       352

    accuracy                           0.95       700
   macro avg       0.95      0.95      0.95       700
weighted avg       0.95      0.95      0.95       700

 Final Validation Accuracy: 94.86%
```








```
--- classification_report
              precision    recall  f1-score   support

          C1       0.95      0.97      0.96       348
          C2       0.97      0.95      0.96       352

    accuracy                           0.96       700
   macro avg       0.96      0.96      0.96       700
weighted avg       0.96      0.96      0.96       700

 Final Validation Accuracy: 96.29%
```



When the input features were **not standardized**, the model had more difficulty learning. The training process was unstable, the **accuracy** was lower (about **94.9%),** and the decision boundary was irregular and noisy.

After **standardizing** the features, the model performed much better. The learning became smoother, the validation accuracy increased to about **96.3%,** and the number of classification errors decreased. The decision boundary also became cleaner and followed the shape of the interlocking circles more accurately.

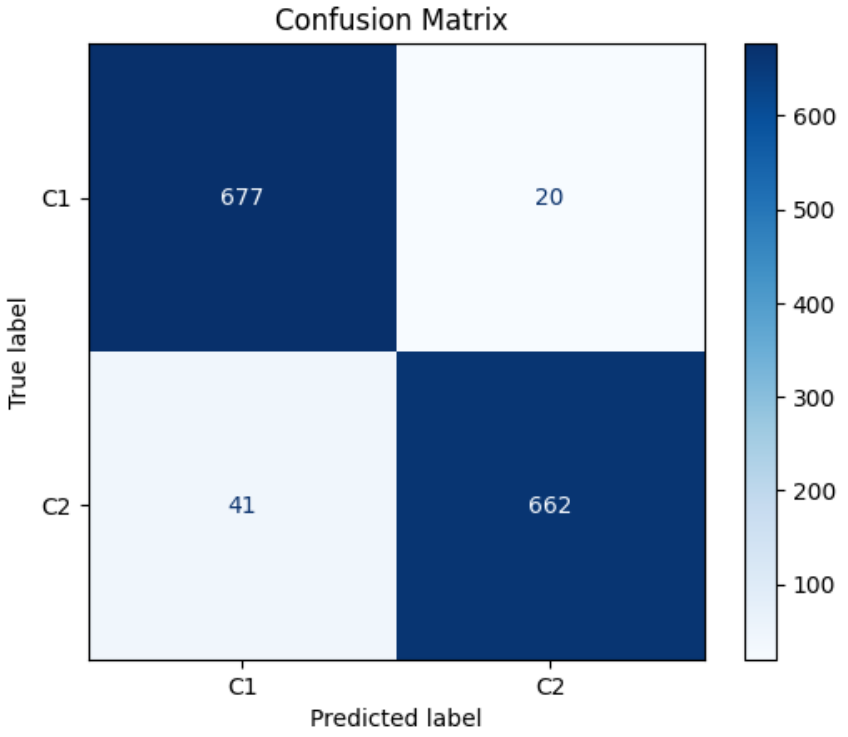
Overall, this shows that feature standardization is very important for CPN. It helps the network learn more stably and improves the final classification accuracy.

Phase 5: Comparative Evaluation (CPN vs. BPN):

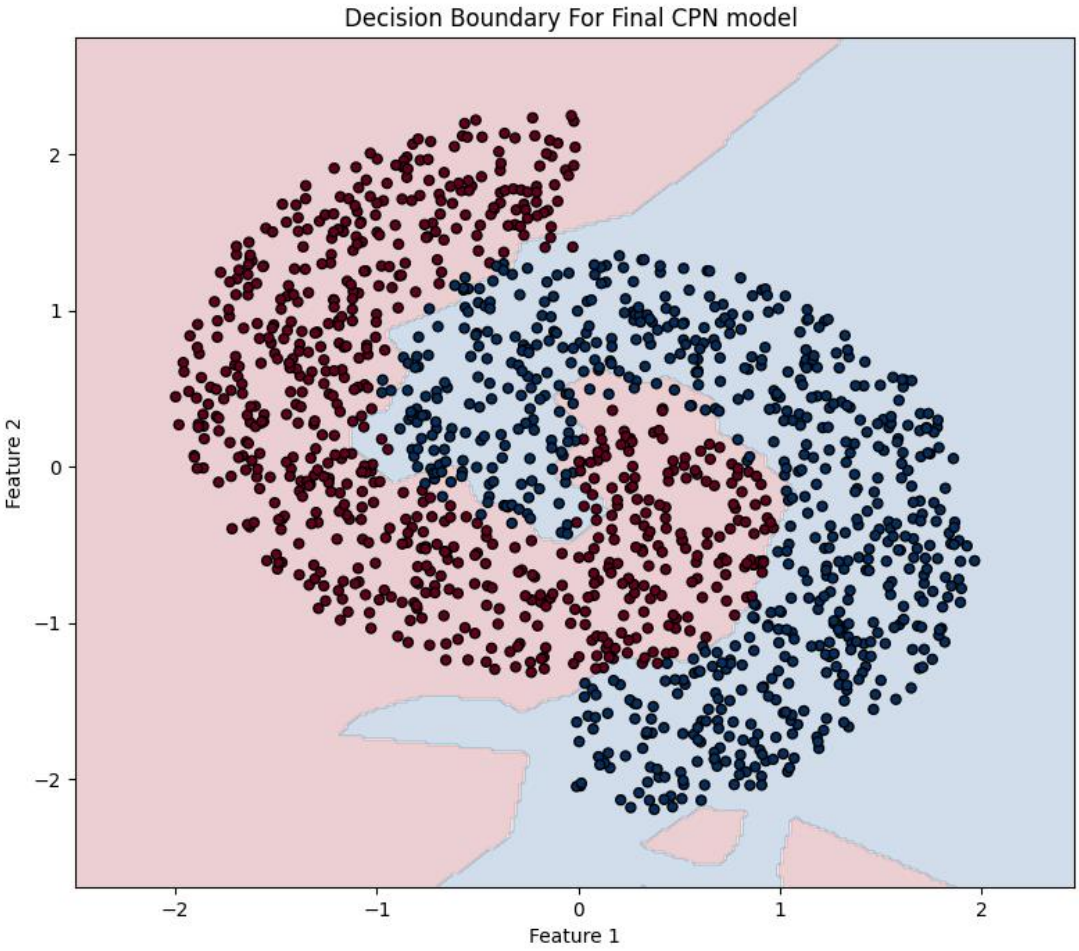
This section compares the performance of the Counter-Propagation Network (CPN) and the Back-Propagation Network (BPN) on the interlocking circles dataset.

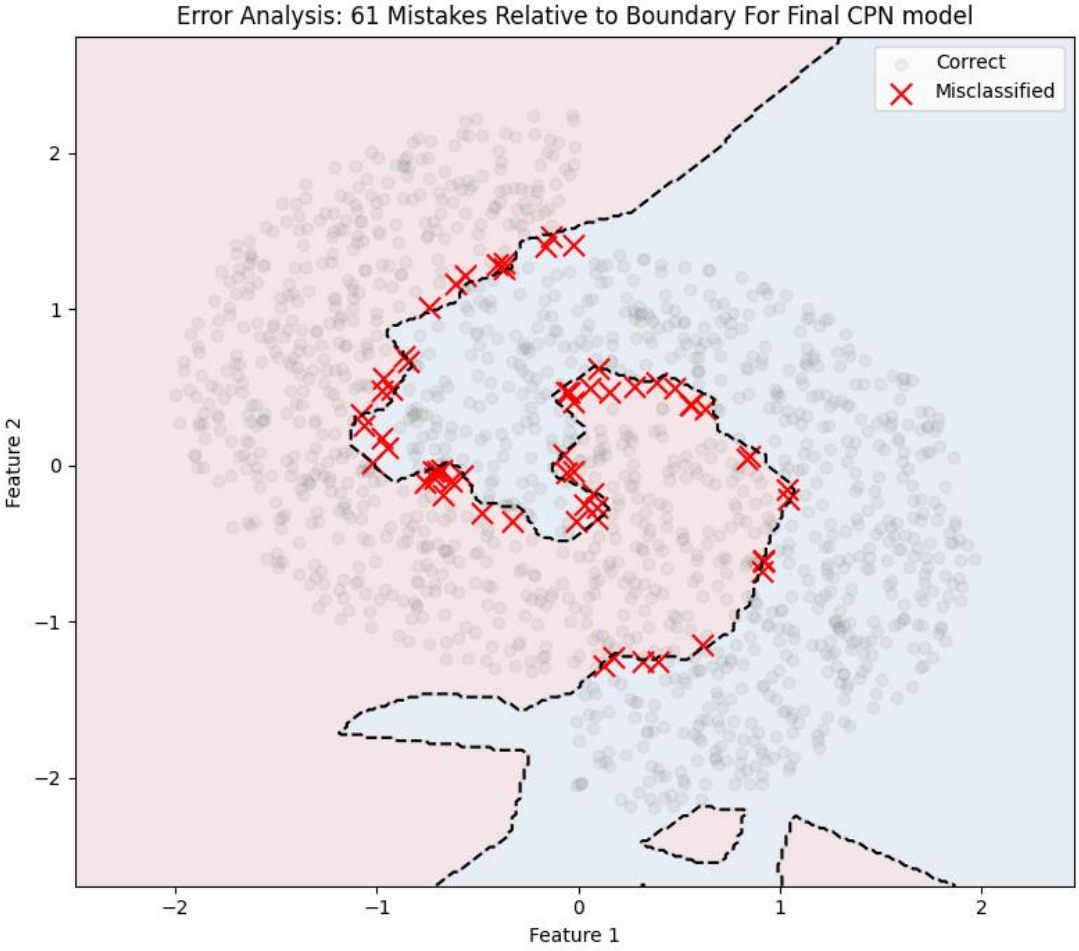
Counter-Propagation Network (CPN):

Fix Parameters and hyperparameters CPN:	Values:
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Total Data Size	7,000
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization
Learning Rate α	0.01
Learning Rate β	0.1
Weight Initialization	competitive sampling



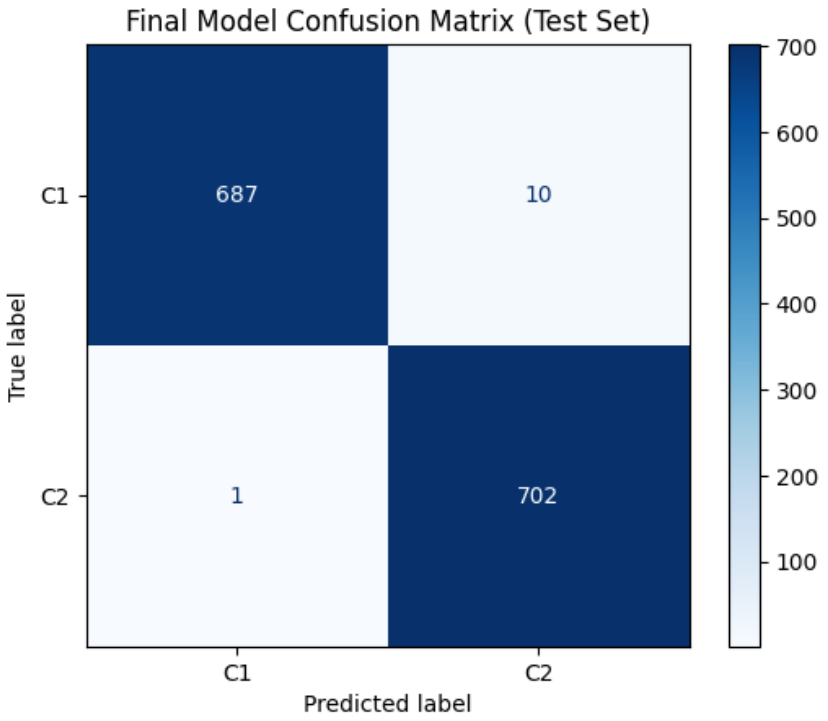

```
--- classification_report
              precision    recall  f1-score   support

          C1       0.94      0.97      0.96       697
          C2       0.97      0.94      0.96       703

    accuracy                           0.96      1400
   macro avg       0.96      0.96      0.96      1400
weighted avg       0.96      0.96      0.96      1400

 Final Validation Accuracy: 95.64%
```


Decision Boundary For Final CPN model
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2


Error Analysis: 61 Mistakes Relative to Boundary For Final CPN model
Correct
Misclassified
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2

Back-Propagation Network (BPN):

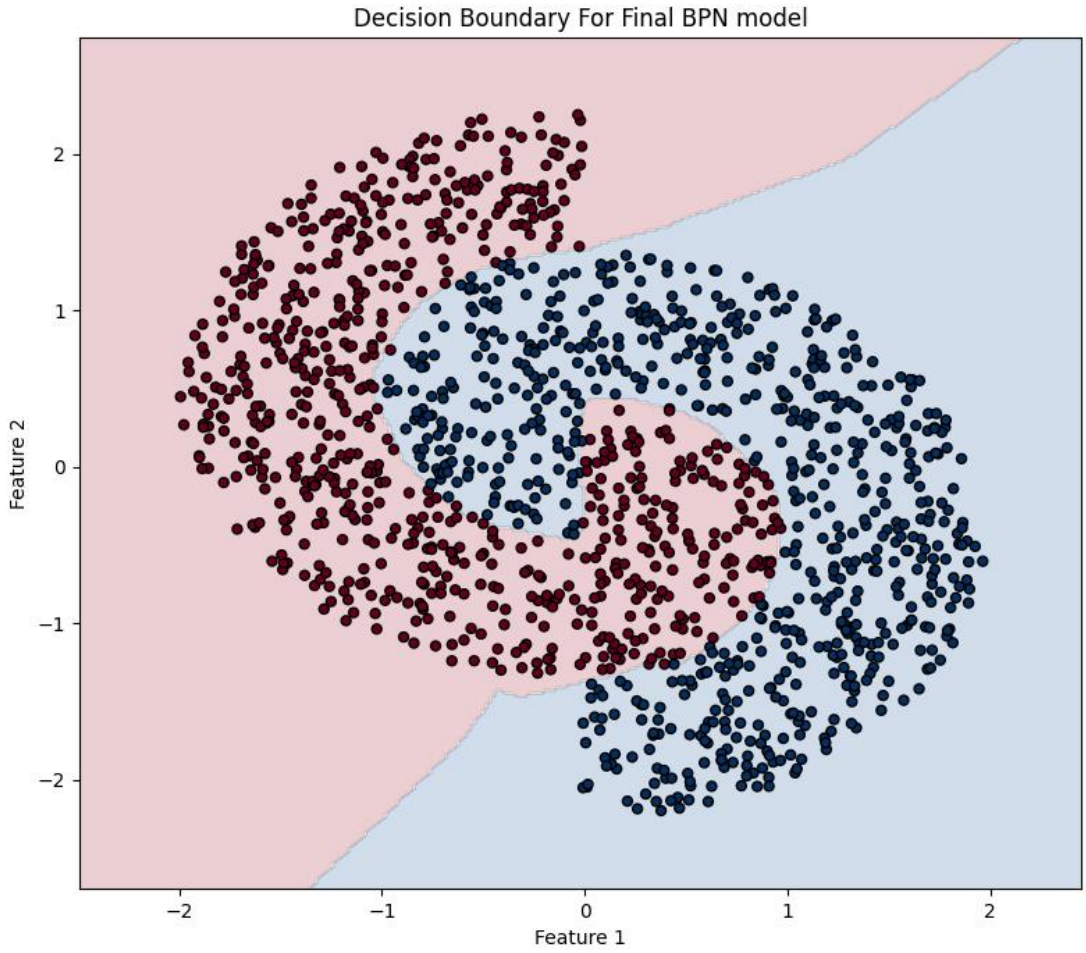
Fix Parameters and hyperparameters BPN:	Values:
Number of Hidden Layers	3
Number of neurons in each layer	(20,15,15)
Total Data Size	7,000
Validation Data Rato	10%
Activation	ReLU
Epoch	150
Feature Scaling	Standardization
Learning Rate η	0.01
Momentum	0.9

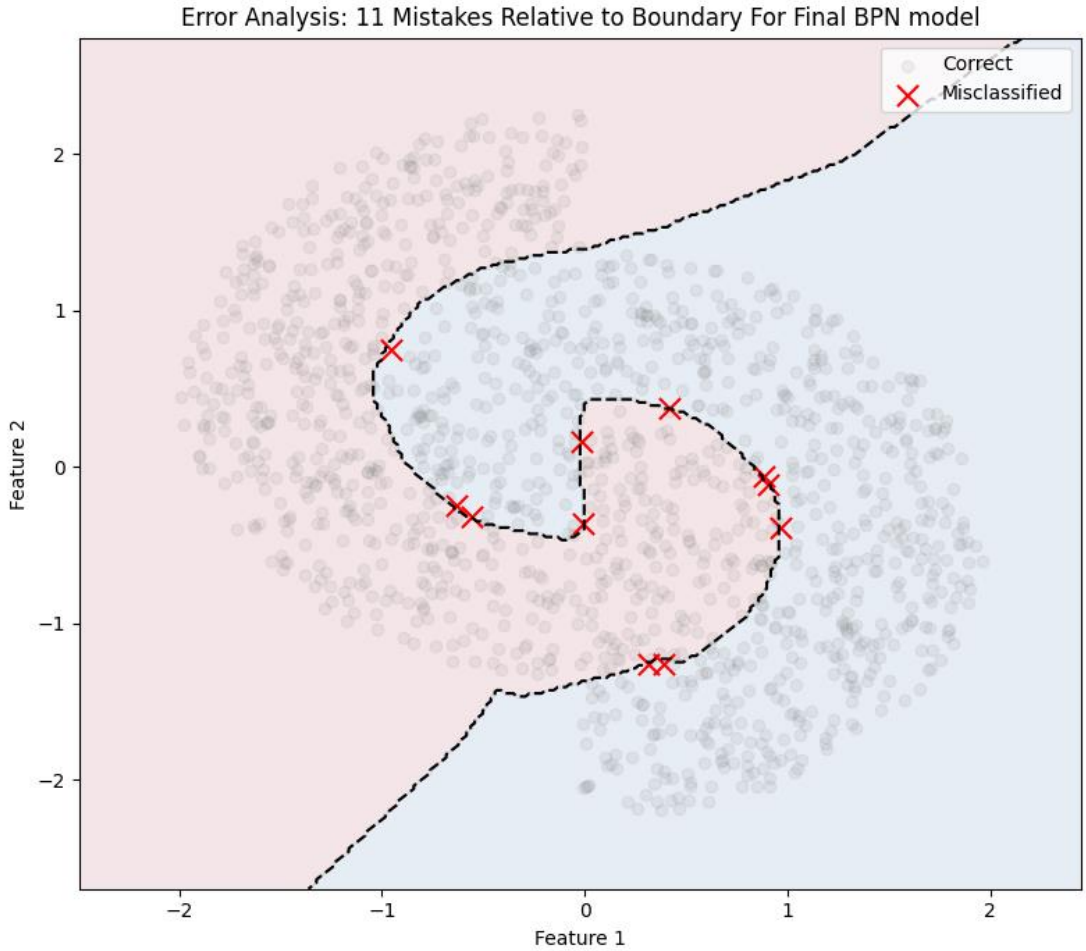


Final Classification Report:

	precision	recall	f1-score	support
Class C1	1.00	0.99	0.99	697
Class C2	0.99	1.00	0.99	703
accuracy			0.99	1400
macro avg	0.99	0.99	0.99	1400
weighted avg	0.99	0.99	0.99	1400

Final Test Accuracy: 99.21%


Decision Boundary For Final BPN model
Feature 1
Feature 2
2
1
0
−1
−2
−2
−1
0
1
2


Error Analysis: 11 Mistakes Relative to Boundary For Final BPN model
Correct
Misclassified
Feature 1
Feature 2
2
1
0
−1
−2
−2
−1
0
1
2

The **CPN model** achieved a validation accuracy of **95.64%** with **61 total errors**. Its decision boundary is relatively rough and block-like because it classifies data using discrete regions formed by competitive learning. However, CPN trains very **fast**.

The **BPN model** performed significantly better, achieving a validation **accuracy** of **99.21%** with **only 11 errors**. Its **decision boundary** is **smooth** and closely follows the shape of the circles, especially in overlapping regions. This is due to the use of gradient-based learning, which allows more precise updates.

Overall, **while CPN is efficient and fast**, **BPN is more suitable** for this classification task because it provides higher accuracy and far fewer errors**, resulting in a 3.5% higher accuracy and nearly 6 times fewer errors than the CPN.**

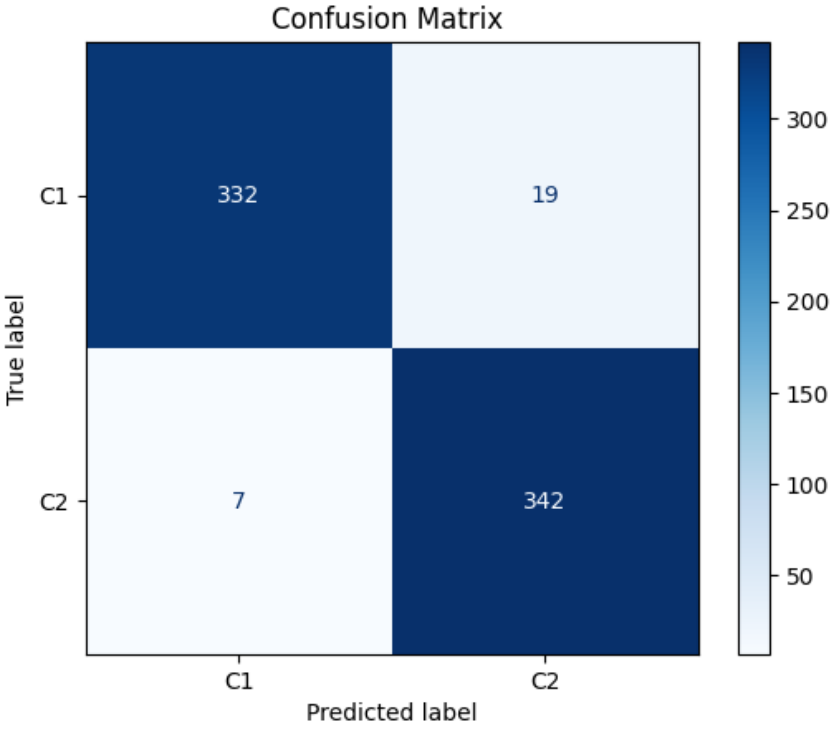
Phase 6: Scalability & Robustness Study

This section evaluates the Counter-Propagation Network's (CPN) ability to generalize across different data volumes, testing its scalability and robustness when handling smaller and larger datasets.

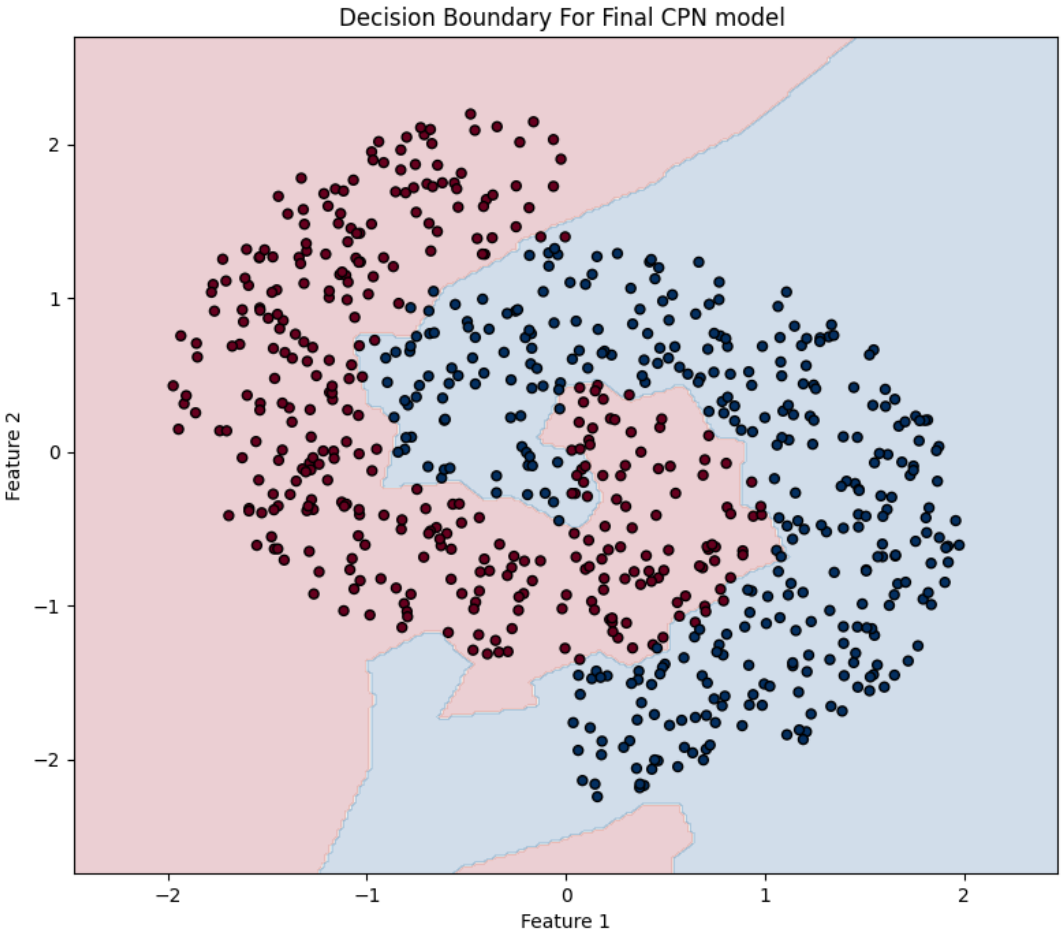
Experiment 1 :

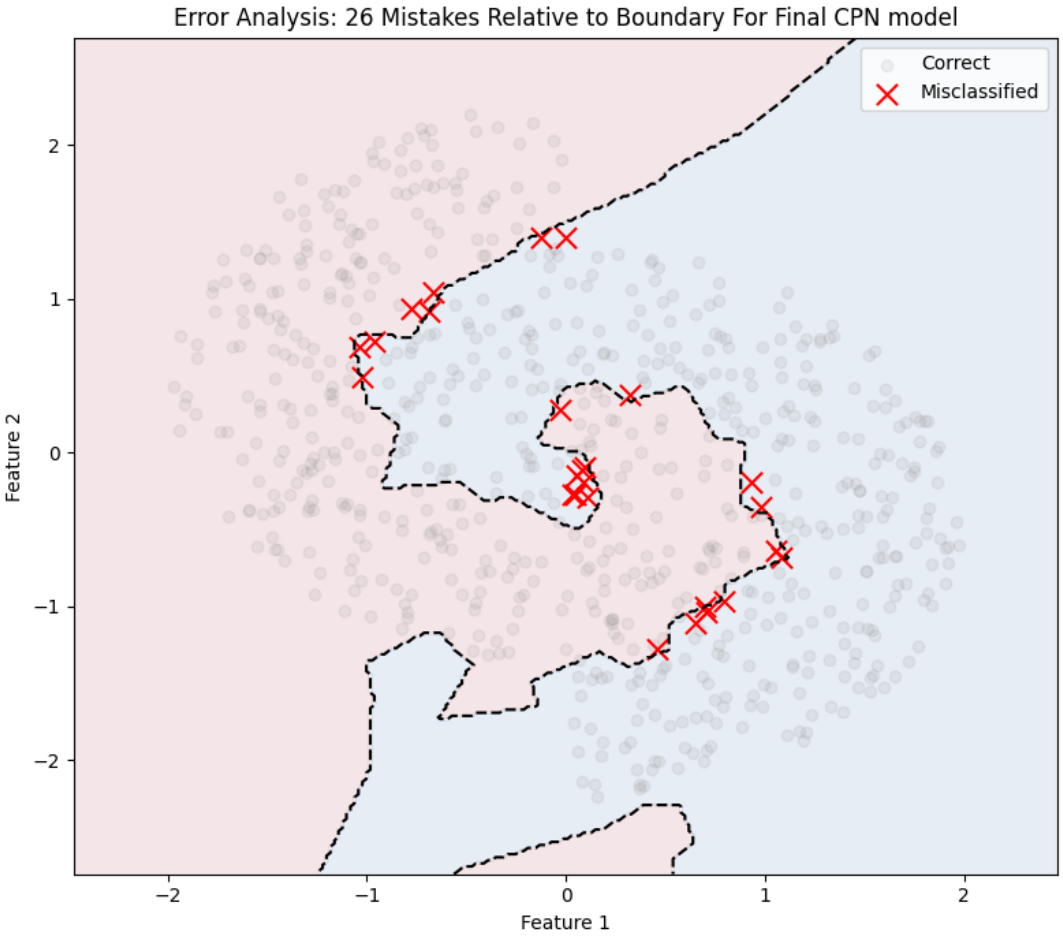
Evaluating the final model's performance on **smaller** Dataset Volume**: 3,500 samples.**

Fix Parameters and hyperparameters CPN:	**Values:**
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization
Learning Rate α	0.01
Learning Rate β	0.1
Weight Initialization	competitive sampling




```
--- classification_report
              precision    recall  f1-score   support

          C1       0.98      0.95      0.96       351
          C2       0.95      0.98      0.96       349

    accuracy                           0.96       700
   macro avg       0.96      0.96      0.96       700
weighted avg       0.96      0.96      0.96       700

 Final Validation Accuracy: 96.29%
```

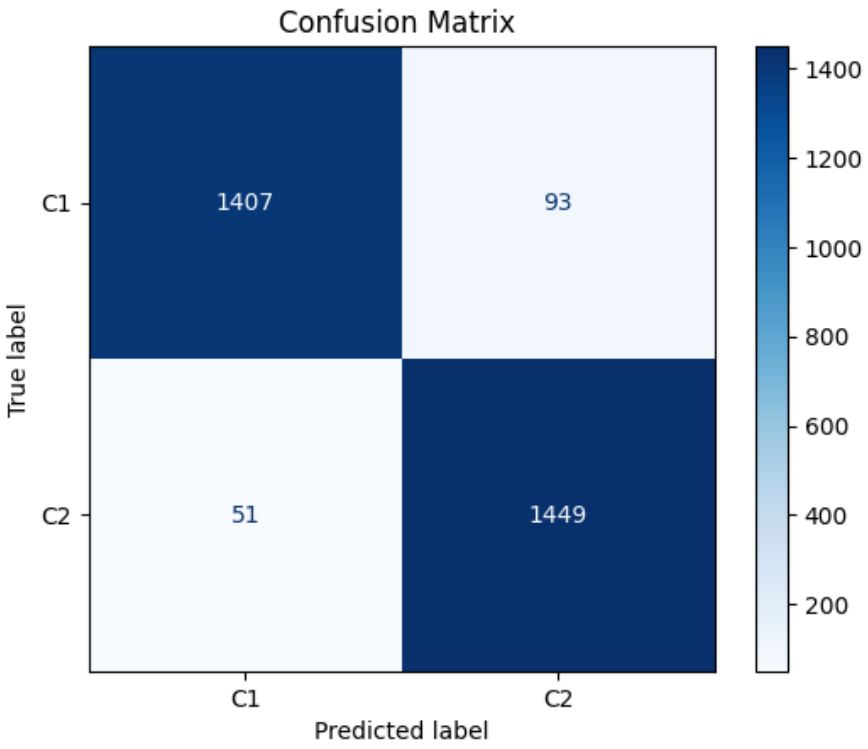

Decision Boundary For Final CPN model
Feature 1
Feature 2
2
1
0
−1
−2
−2
−1
0
1
2


Error Analysis: 26 Mistakes Relative to Boundary For Final CPN model
Correct
Misclassified
Feature 1
Feature 2
2
1
0
−1
−2
−2
−1
0
1
2

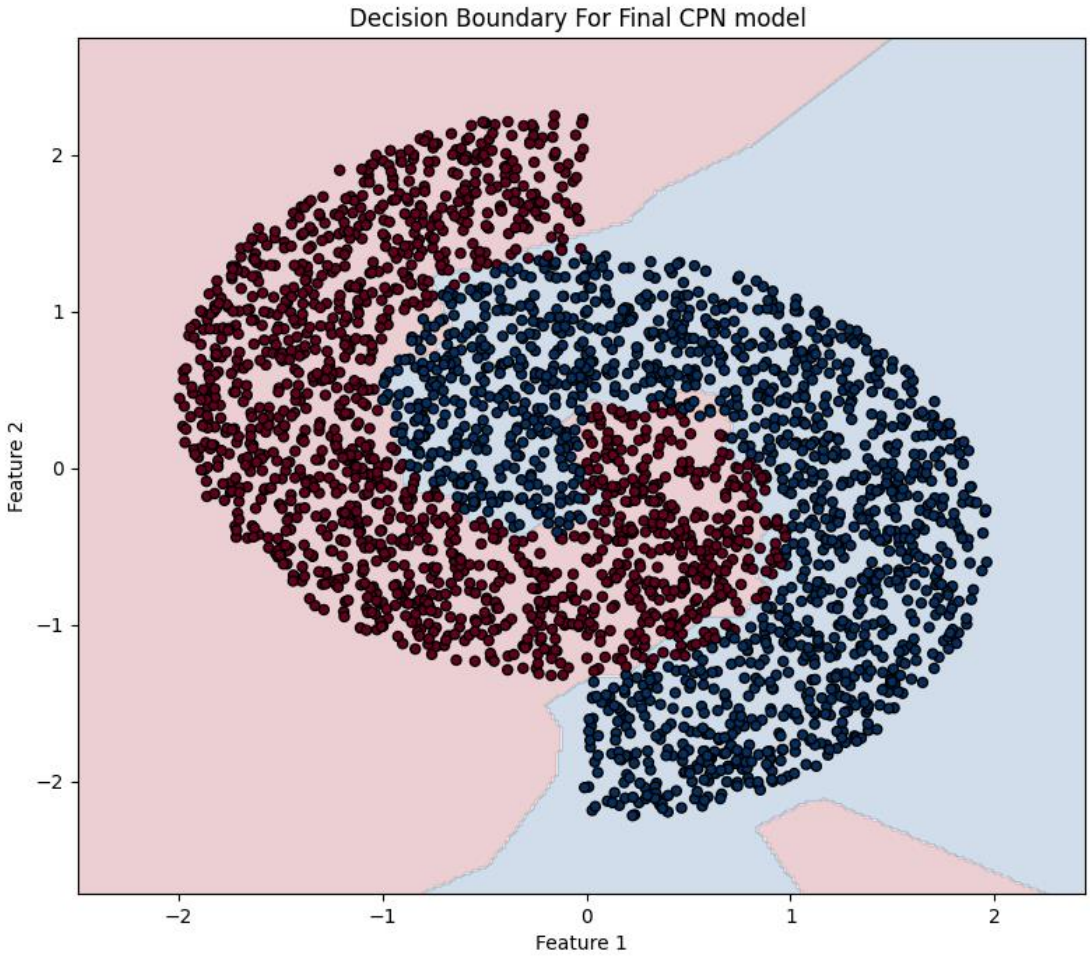
Experiment 2 :

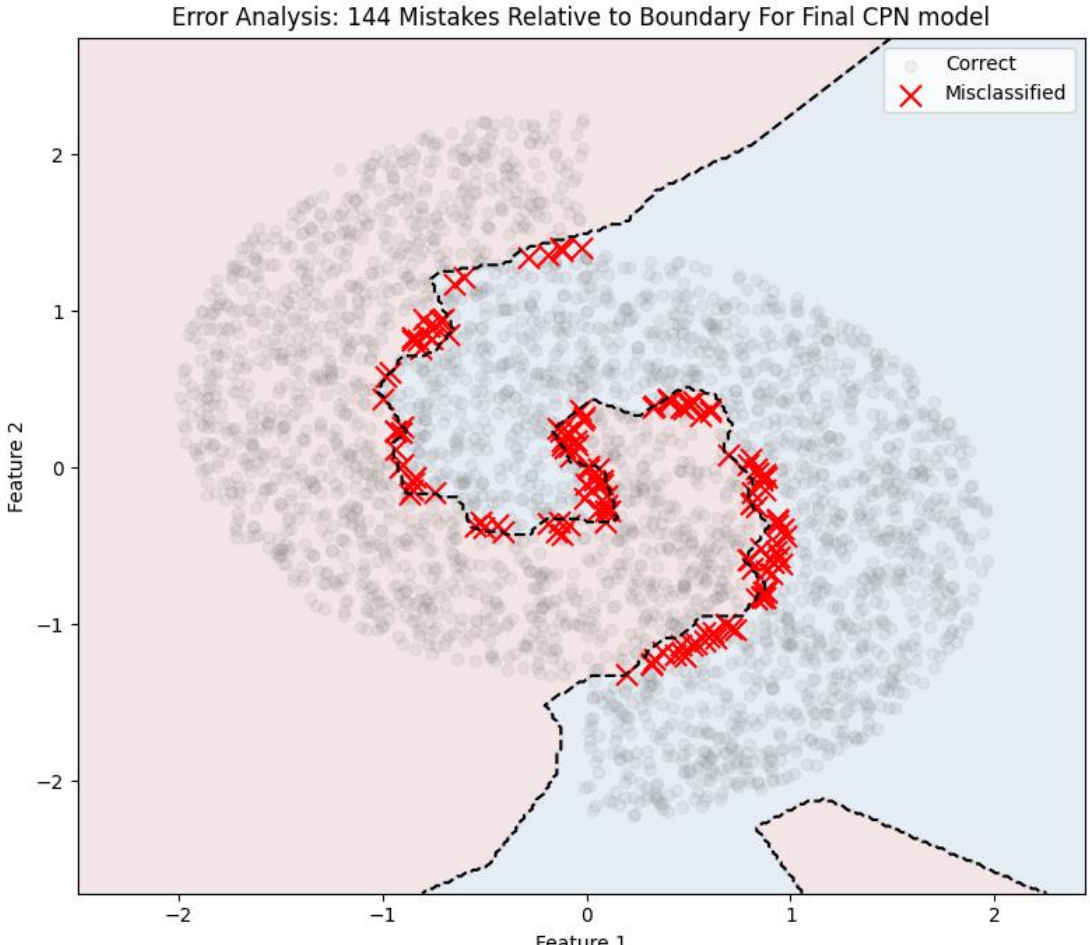
Evaluating the model's performance on larger dataset Volume:**15,000 samples.**

Fix Parameters and hyperparameters CPN:	**Values:**
Number of Hidden Layers	1
Neurons in Kohonen Layer	100
Validation Data Rato	10%
Distance Metric	Euclidean Distance
Epoch	150
Feature Scaling	Standardization
Learning Rate α	0.01
Learning Rate β	0.1
Weight Initialization	competitive sampling




```
--- classification_report
              precision    recall  f1-score   support

          C1       0.97      0.94      0.95      1500
          C2       0.94      0.97      0.95      1500

    accuracy                           0.95      3000
   macro avg       0.95      0.95      0.95      3000
weighted avg       0.95      0.95      0.95      3000

Final Validation Accuracy: 95.20%
```


Decision Boundary For Final CPN model
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2


Error Analysis: 144 Mistakes Relative to Boundary For Final CPN model
Correct
Misclassified
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2

With training the model on a **smaller dataset (3,500 samples),** the CPN achieved an **accuracy** of **96.29%**. The **confusion matrix** shows **26 classification errors (19 in Class 1 and 7 in Class 2).** The model produced a good decision boundary that followed the shape of the data. These results show that the model remains robust even with limited data and performs almost well .

When the **dataset size** was increased to **15,000 samples**, the model maintained a similar accuracy of **95.20%**, with only a small decrease. The total number of errors increased to **144 (93 in Class 1 and 51 in Class 2**), which is expected because the dataset is much larger. However, the overall structure of the decision boundary remained consistent.

Overall, the results show that the CPN is both **robust and scalable**. It can handle increases or decreases in data size with only a small drop in accuracy, which means the model can generalize well across different dataset sizes.

Conclusion

This project focused on implementing and evaluating a Counter-Propagation Network (CPN) for the classification of a complex interlocking circles dataset. Several experimental phases were conducted to analyze the effects of network architecture, weight initialization, feature scaling, model comparison, and scalability.

Structural Optimization: The experiments showed that using **100 neurons in the Kohonen layer** works best for capturing the complex structure of the data. When fewer neurons were used, the model could not represent the decision boundary as accurately. In addition, using small learning rates with decay ($\alpha = 0.01$ and $\beta = 0.1$) helped keep the training process stable, especially in the later epochs.

Impact of Initialization: Using the weight initialization has a big effect on the model's performance. Initializing the Kohonen weights using **Sampling from the training data** worked much better than random initialization. This approach helped the model converge faster and also avoided the dead neuron problem.

The Necessity of Standardization: feature standardization played a key role. When the input data was not standardized, the model became unstable and made more classification errors. After applying standardization, the training became smoother and the accuracy clearly improved.

CPN vs. BPN Comparison: When compared with the Back-Propagation Network (BPN), the BPN achieved higher accuracy **99.21%**, but the CPN was faster , computationally efficient and also still performed well when properly configured.

Scalability: The scalability tests showed that the CPN is robust. Even when the dataset size increased significantly, the model kept a high accuracy, which shows that it can handle larger datasets reliably.

Overall, this project shows that CPN can be an effective model for non-linear classification if the data is standardized, the network size is chosen correctly, and the weights are initialized properly.

Strengths and Limitations of the CPN Model

Strengths

Fast Training: One of the main advantages of the CPN model is that it trains much faster than the Back-Propagation Network (BPN) used in the previous project.

Efficient Initialization: When Competitive Sampling is used to initialize the weights, the model starts with relatively high accuracy from the beginning. This makes the training process faster and more efficient.

Scalability: The CPN architecture is also quite robust. The experiments showed that the model keeps stable performance even when the dataset size increases from 3,500 to 15,000 samples.

Limitations

Lower Accuracy: Although the CPN performs well, its final accuracy (95.64%) is still lower than the BPN model, which achieved 99.21%.

Jagged Decision Boundaries: Unlike BPN, which produces smooth boundaries, the CPN boundaries look more jagged or block-like. This happens because they are formed by discrete Kohonen neurons.

Sensitive Initialization: The model is very sensitive to how the weights are initialized. If a large random range is used, some neurons may never win during training. These are called *dead neurons* and they can increase the error rate.

Dependence on Feature Scaling: The CPN model also requires the appropriate feature scaling technique according to problem. Without scaling, the Euclidean distance calculations become biased, which makes it harder for the network to learn the correct structure of the data.